

**TECHNICAL UNIVERSITY OF SOFIA**

**FACULTY OF COMPUTER SYSTEMS AND TECHNOLOGIES**

**DESIGN AND IMPLEMENTATION OF A C++  
COMPILE-TIME OBJECT-RELATIONAL  
MAPPING LIBRARY**

**DIPLOMA THESIS**

**Author:**

Alex Tsvetanov, Faculty No. 121222225

Specialisation: Computer and Software Engineering

**Supervisor:**

Assoc. Prof. Eng. Galya Pavlova, Ph.D.

Sofia, 2026

# CONTENTS

|             |                                                                                                           |           |
|-------------|-----------------------------------------------------------------------------------------------------------|-----------|
| <b>I.</b>   | <b>Introduction . . . . .</b>                                                                             | <b>3</b>  |
| 1.1.        | Problem Statement . . . . .                                                                               | 3         |
| 1.2.        | Objectives and Tasks . . . . .                                                                            | 3         |
| 1.3.        | Scope and Limitations . . . . .                                                                           | 4         |
| 1.4.        | Thesis Structure . . . . .                                                                                | 4         |
| <b>II.</b>  | <b>Review of Existing Solutions . . . . .</b>                                                             | <b>6</b>  |
| 2.1.        | Classification of approaches . . . . .                                                                    | 6         |
| 2.2.        | Evaluation criteria . . . . .                                                                             | 8         |
| 2.3.        | Native client libraries . . . . .                                                                         | 8         |
| 2.4.        | Runtime query builders . . . . .                                                                          | 9         |
| 2.5.        | Typed SQL DSL solutions . . . . .                                                                         | 9         |
| 2.6.        | Classical ORM frameworks and code generation . . . . .                                                    | 10        |
| 2.7.        | Limits of relationally centred abstractions . . . . .                                                     | 10        |
| 2.8.        | Positioning of the present work . . . . .                                                                 | 11        |
| 2.9.        | Conclusions from the comparative analysis . . . . .                                                       | 12        |
| <b>III.</b> | <b>Theoretical Foundations of Compile-time Object-Relational Mapping and<br/>Storage Models . . . . .</b> | <b>13</b> |
| 3.1.        | Taxonomy of the problem domain . . . . .                                                                  | 13        |
| 3.2.        | Compile-time Object-Relational Mapping as an architectural approach . . . . .                             | 14        |
| 3.3.        | Static data modelling in C++ . . . . .                                                                    | 15        |
| 3.4.        | Typed query IR . . . . .                                                                                  | 17        |
| 3.5.        | Relationship theory and storage strategies . . . . .                                                      | 18        |
| 3.6.        | Relational model . . . . .                                                                                | 19        |
| 3.7.        | Document model . . . . .                                                                                  | 20        |
| 3.8.        | Key-value model . . . . .                                                                                 | 20        |
| 3.9.        | Graph model . . . . .                                                                                     | 20        |
| 3.10.       | Wide-column model . . . . .                                                                               | 21        |
| 3.11.       | Test evidence behind the theoretical claims . . . . .                                                     | 21        |
| 3.12.       | Boundaries of the shared abstraction . . . . .                                                            | 22        |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>IV. Library Architecture</b>                                             | <b>23</b> |
| 4.1. Architectural goals and constraints                                    | 23        |
| 4.2. Layered organisation and responsibility separation                     | 24        |
| 4.3. The entity layer as architectural foundation                           | 25        |
| 4.4. Query IR as the central architectural component                        | 26        |
| 4.5. The connector layer and the formal contract                            | 26        |
| 4.6. The user-facing db<DB> facade                                          | 28        |
| 4.7. Migrations as an architectural extension                               | 29        |
| 4.8. Architectural invariants                                               | 29        |
| 4.9. Architectural conclusion                                               | 30        |
| <b>V. Implementation of the Core Subsystems</b>                             | <b>31</b> |
| 5.1. Repository-backed mini case study: User and Post                       | 31        |
| 5.2. Scalar properties and naming of storage containers                     | 32        |
| 5.3. Field wrappers, rules, and placeholders                                | 32        |
| 5.4. Construction of query objects                                          | 33        |
| 5.5. Execution, results, and field access                                   | 34        |
| 5.6. Prepared queries as a reusable execution artefact                      | 34        |
| 5.7. Migration layer and DDL generation                                     | 35        |
| 5.8. Thread safety and transaction helpers                                  | 36        |
| 5.9. Implementation conclusion                                              | 38        |
| <b>VI. Backend Scenarios Across Relational and Non-relational Databases</b> | <b>39</b> |
| 6.1. Methodology of the analysis                                            | 39        |
| 6.2. Relational baseline scenario: SQLite                                   | 40        |
| 6.3. Relational portability: PostgreSQL and MySQL                           | 41        |
| 6.4. Document scenario: MongoDB                                             | 41        |
| 6.5. Key-value scenario: Redis                                              | 42        |
| 6.6. Graph scenario: Neo4j                                                  | 42        |
| 6.7. Wide-column scenario: Cassandra                                        | 43        |
| 6.8. Comparison between test evidence and backend contract                  | 44        |
| 6.9. Comparative synthesis                                                  | 45        |
| <b>VII. Verification and Extensibility Through New Connectors</b>           | <b>46</b> |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| 7.1. Formal contract of the connector layer . . . . .                        | 46        |
| 7.2. Mandatory elements of a minimal connector . . . . .                     | 47        |
| 7.3. Typing, signatures, and naming . . . . .                                | 48        |
| 7.4. Capability mechanism and compile-time restrictions . . . . .            | 49        |
| 7.5. Minimal connector skeleton and operational connector skeleton . . . . . | 49        |
| 7.6. Transactions, thread safety, and prepared queries . . . . .             | 51        |
| 7.7. Migrations and schema-aware integration . . . . .                       | 51        |
| 7.8. Methodology for adding a new backend . . . . .                          | 52        |
| 7.9. Criteria for a “fully implemented and functioning” connector . . . . .  | 53        |
| 7.10. Verification through tests . . . . .                                   | 53        |
| 7.11. Assessment of the existing connectors . . . . .                        | 54        |
| <b>VIII. Limitations and Future Work . . . . .</b>                           | <b>55</b> |
| 8.1. Strategic directions of development . . . . .                           | 55        |
| 8.2. Broader live backend coverage . . . . .                                 | 56        |
| 8.3. Full schema-aware integration and runtime introspection . . . . .       | 56        |
| 8.4. Thread safety and connection lifecycle . . . . .                        | 57        |
| 8.5. Low-level execution and wire-protocol optimisation . . . . .            | 58        |
| 8.6. Integration with C++26 reflection and further automation . . . . .      | 58        |
| 8.7. Extension of the connector contract and capability taxonomy . . . . .   | 59        |
| 8.8. Need for stricter empirical evaluation . . . . .                        | 60        |
| 8.9. Summary . . . . .                                                       | 60        |
| <b>IX. Conclusion . . . . .</b>                                              | <b>61</b> |
| 9.1. Summary of the solved problem . . . . .                                 | 61        |
| 9.2. Summary of achieved results . . . . .                                   | 62        |
| 9.3. Scientific and practical contributions . . . . .                        | 63        |
| 9.4. Principal limitations and the boundaries of generalisation . . . . .    | 64        |
| 9.5. Significance for practice and for future research . . . . .             | 65        |
| 9.6. Closing remarks . . . . .                                               | 65        |
| <b>X. Glossary of Abbreviations and Terms . . . . .</b>                      | <b>67</b> |
| 10.1. Basic abbreviations . . . . .                                          | 67        |
| 10.2. Working definitions of key concepts . . . . .                          | 68        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| 10.3. Notation and naming used in the text . . . . .                          | 69        |
| 10.4. Repository context of the most frequently cited artefacts . . . . .     | 70        |
| 10.5. Conclusion . . . . .                                                    | 71        |
| <b>XI. Map of Verification Scenarios and Repository Artefacts . . . . .</b>   | <b>72</b> |
| 11.1. Overall structure of the evidence layer . . . . .                       | 72        |
| 11.2. Catalogue of the principal unit tests . . . . .                         | 73        |
| 11.3. Integration and live tests . . . . .                                    | 74        |
| 11.4. Capability evidence by backend family . . . . .                         | 75        |
| 11.5. CMake registration as part of the verification design . . . . .         | 76        |
| 11.6. Conclusions from the appendix . . . . .                                 | 77        |
| <b>XII. Atlas of Capability Profiles and Backend Maturity . . . . .</b>       | <b>79</b> |
| 12.1. The capability mechanism as a formal layer . . . . .                    | 79        |
| 12.2. Connector maturity as a ladder rather than a binary judgement . . . . . | 80        |
| 12.3. Capability profiles by backend family . . . . .                         | 81        |
| 12.4. Maturity map by available test evidence . . . . .                       | 83        |
| 12.5. What the capability atlas reveals about the architecture . . . . .      | 85        |
| 12.6. Conclusion . . . . .                                                    | 85        |

## ABSTRACT

This diploma thesis presents the design, implementation, and evaluation of an Compile-time Object-Relational Mapping library for C++23 [6]. The central idea of the work is that the logical structure of queries and the logical structure of the data model should not be described through runtime strings, but through typed `constexpr` objects validated by the compiler. In this way, a substantial class of traditional data-access errors — incorrect field names, incompatible predicate types, missing query parts, and misuse of unsupported backend operations — is moved from execution time to compilation time.

The developed library uses a static data model expressed in C++ through constructions such as `property<T,Name>`, `relationship<...>`, `table_name_trait<T>`, and a typed query IR represented by `select_query`, `insert_query`, `update_query`, and `delete_query`. This intermediate model is independent of the concrete database type. Materialisation toward relational and non-relational systems is performed by specialised `connector_trait<DB>` implementations, which declare supported capabilities and translate the same logical query into SQL, BSON-like filters, Redis commands, Cypher, or CQL depending on the selected backend.

The thesis analyses both relational scenarios for SQLite, PostgreSQL, and MySQL, and non-relational scenarios for MongoDB, Redis, Neo4j, and Cassandra. It demonstrates how the same C++ model dictates the logical schema while being materialised differently as a table, document, key-value structure, graph node, or wide-column row. Particular attention is given to the role of `store_as::reference` and `store_as::embed` as a shared abstraction for object relationships whose concrete interpretation depends on the storage model.

Validation of the work is grounded in the repository’s unit and integration tests. These tests cover query IR construction, capability-based restriction of operations, prepared queries, migrations, thread safety, and the behaviour of individual connectors with either live or mock drivers. This makes it possible to formulate not only an architectural model, but also concrete criteria for a “fully implemented and functioning” connector.

The main contribution of the thesis lies in three directions. First, it proposes a practically usable model for an Compile-time Object-Relational Mapping library that combines static type safety with a backend-independent architecture. Second, it shows how a C++ data model can serve as the primary source of a logical schema across different classes of databases. Third, it

formalises a connector contract through which the library can be extended with new backend implementations without changing the user-facing query API.

**Keywords:** C++23, Compile-time Object-Relational Mapping, query IR, `connector_trait`, type safety, relational and non-relational databases, prepared queries, migrations

# I. INTRODUCTION

## 1.1. Problem Statement

Data access remains one of the areas where the mismatch between the programming language and the storage model produces the greatest number of structural errors. In traditional database access layers, queries are described as strings and validated only at execution time. This means that incorrect column names, incompatible types, invalid JOIN constructs, or wrongly supplied parameters are discovered late — often against a real database and only after additional manual testing. At the same time, switching from one backend to another usually requires rewriting the entire data-access layer.

Object-relational mapping is the classical way of connecting an application’s object model with persistent storage [2]. Most ORM implementations, however, remain strongly tied to runtime mechanisms: SQL strings, code generation, annotations, macros, or dynamic metadata descriptions. In the context of C++, this introduces an additional tension. The language provides a powerful type system, templates, `constexpr` evaluation, and `constexpr`, yet database libraries rarely exploit these mechanisms far enough for the compiler to become a first-class validator of query structure and backend compatibility.

A distinctive feature of the present system is that, despite the name Compile-time Object-Relational Mapping library, its architecture is not restricted to the relational model alone. Instead, it uses the same C++ data model and the same query IR as a logical layer over relational and non-relational systems. This raises a second research question: which parts of the ORM abstraction are genuinely universal, and which parts must remain explicitly constrained through a capability-based backend contract.

## 1.2. Objectives and Tasks

The primary objective of the thesis is to present the design and implementation of a Compile-time Object-Relational Mapping library for C++ that:

1. models data and queries through statically typed C++ constructs;
2. allows SELECT, INSERT, UPDATE, and DELETE queries to be built as `constexpr` objects;
3. separates logical query structure from backend-specific materialisation;



4. supports both relational and non-relational connectors through `connector_trait<DB>`;
5. uses compile-time capability tags to reject unsupported operations;
6. demonstrates how the C++ model determines both the logical schema and the physical data structure for different backend families;
7. formalises a contract for adding new connectors;
8. validates the architecture through systematic unit and integration tests.

To achieve that objective, the work performs the following tasks: analysis of existing solutions; definition of a static entity model; construction of a typed query IR; design of the connector layer and capability mechanism; implementation of connectors for different classes of databases; investigation of migrations, prepared queries, and thread safety; and formulation of extensibility criteria.

### 1.3. Scope and Limitations

The thesis treats the library as a modelling, query-construction, and execution layer. Its scope therefore includes the entity model, query IR, placeholder mechanism, prepared queries, migration model, connector trait architecture, capability gating, and the concrete backend implementations present in the repository. A substantial part of the evaluation is the scenario analysis of SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j, and Cassandra.

Out of immediate scope remain production-scale optimisations such as fully generalised connection management, backend-complete migration strategies, large-scale benchmarking, and full exploitation of future C++26 reflection. These are discussed as future work in Chapter VIII.

### 1.4. Thesis Structure

The thesis is organised as follows.

**Chapter II** reviews existing C++ solutions for database access and ORM, and positions the present work with respect to them.

**Chapter III** presents the theoretical foundations of Compile-time Object-Relational Mapping and the storage models required for the later relational and non-relational analysis.

**Chapter IV** describes the architecture of the library: entity declarations, query IR, `connector_trait`, capability mechanisms, and the place of migrations within the overall design.

**Chapter V** follows the implementation of the core subsystems and shows how they connect at the code and runtime levels.

**Chapter VI** analyses how the same logical abstraction is materialised in different backends — relational, document, key-value, graph, and wide-column.

**Chapter VII** examines the verification strategy and the formal contract for extending the library with new connectors.

**Chapter VIII** outlines the main limitations and future development directions, while **Chapter IX** summarises the results and contributions.

## II. REVIEW OF EXISTING SOLUTIONS

Reviewing existing solutions is necessary in order to evaluate whether the proposed architecture delivers a genuine contribution or merely reformulates already known practices. In the space of C++ database libraries, two orthogonal axes are especially useful: the level of abstraction and the stage at which queries are constructed and validated. The abstraction level ranges from almost direct use of a client C API to fully fledged ORM behaviour. The validation stage may be runtime, hybrid, or compile time.

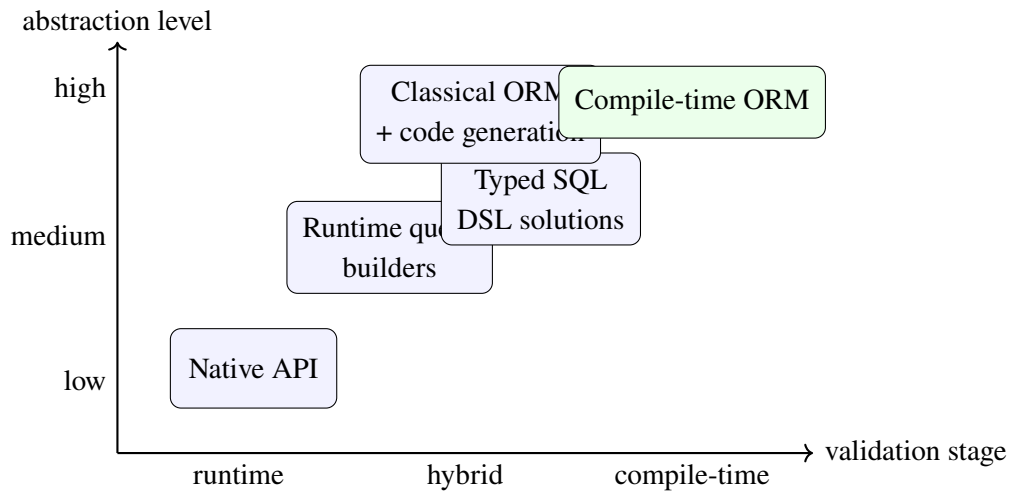
These two axes are especially important in the context of the present thesis. If one observes only “API convenience”, it is easy to miss the essential difference between a library that merely makes SQL strings more readable and a library that truly transfers structural correctness into the compiler. If, on the other hand, one looks only at when an error appears, one may underestimate the cost of abstraction, build-toolchain complexity, and suitability for more than one storage model.

### 2.1. Classification of approaches

At the lowest level stand native client libraries such as the SQLiteC API [4], libpq [8], and MySQLConnector/C [7]. They provide maximal control over queries, parameters, and results, but offer no logical abstraction. The application must work directly with SQL strings, column indices, parameter positions, and backend-specific types.

The next group consists of runtime query-builder libraries. These typically improve readability and reduce part of the boilerplate, but they continue to rely on runtime construction of textual query representations. Type safety is only partial and usually stops at the API boundary. Structural validity of the query remains a runtime concern.

At the highest abstraction level lies the ORM approach. There, C++ classes are associated with tables, columns, and relationships, and the library generates the required queries. The benefit is a significant reduction in boilerplate code. The cost is that many such frameworks introduce external code generators, annotations, macros, or opaque runtime metadata, making the correspondence between C++ code and the actual database operation harder to follow.



**Figure 1. Positioning of the principal solution families by abstraction and validation stage**

Figure 1 shows that the compile-time ORM approach is not merely “another ORM library”. It occupies the upper-right quadrant, where high abstraction is combined with early structural validation. That combination is rare because it demands greater architectural discipline inside the library itself.

**Table 1. Comparison of major data-access approaches in C++**

| Approach              | Abstraction level | Validation stage           | Strength                                                             | Primary limitation                         |
|-----------------------|-------------------|----------------------------|----------------------------------------------------------------------|--------------------------------------------|
| Native API            | low               | runtime                    | full control and closeness to the driver                             | strong coupling to a concrete backend      |
| Runtime query builder | medium            | mostly runtime             | a more readable API and less boilerplate                             | the query still materialises as text       |
| Typed SQL DSL         | medium to high    | partial compile-time       | stronger static structure for the SQL world                          | usually remains limited to SQL             |
| Classical ORM         | high              | runtime or code generation | convenient object-mapping behaviour                                  | complex toolchain and reduced transparency |
| Compile-time ORM      | high              | compile time for structure | early discovery of structural errors and a formal connector contract | greater complexity in the template layer   |

## 2.2. Evaluation criteria

For the comparison to be academically complete, it is not enough to say that one library is “more convenient” while another is “lower level”. In the present work, five criteria are used:

1. **Level of logical abstraction** — to what degree the library allows the data model to be described independently of the immediate backend syntax.
2. **Degree of static validation** — which aspects of the model and query can be checked before execution.
3. **Portability across backends** — whether the logical model can be adapted to more than one storage family.
4. **Toolchain complexity** — whether the library requires external code generation, extra preprocessing stages, or generated metadata.
5. **Transparency of errors and execution** — how clearly a developer can follow the link between the C++ source, the generated query, and the runtime behaviour.

These criteria are directly related to the objectives of the thesis. If the proposed library is to be both expressive and formally checkable, then it must improve on alternatives not only in convenience, but in the balance between these five dimensions.

## 2.3. Native client libraries

SQLite [3], libpq, and MySQLConnector/C are representative of the approach in which the application constructs and submits queries directly to the driver. This is the most universal and often the most efficient option, but it places the full burden of correctness on application code. Developers must manually keep field names, parameter ordering, and schema details in sync. When the backend changes, not only the SQL dialect changes, but also the entire connection, preparation, binding, and result-reading API.

This becomes especially problematic when the same domain model must be connected to more than one class of database. In such cases the application accumulates conditional code paths, backend-specific SQL strings, and duplicated hydration logic. Beyond a certain system size, this weakens not only convenience but also confidence in correctness.

The strength of the native-API approach lies in its honesty. It does not promise more abstraction than it actually provides. Precisely for that reason, it often serves as a control point against which higher-level abstractions may be judged.

## **2.4. Runtime query builders**

Libraries such as SOCI [5] and part of the lighter fluent SQL tooling seek a balance between readability and control. They offer a more convenient C++ interface for query construction, yet in many cases they still rely on runtime description and subsequent rendering to SQL. In practice this means that part of the query structure is better encapsulated, but backend-independent switching between fundamentally different storage models remains limited.

For the present work, these libraries are important as a point of comparison. They show that a fluent API and typed building blocks are useful, but not sufficient on their own. The decisive question is whether the compiler can validate the full logical structure of the query and whether the same abstraction can extend beyond the SQL world.

## **2.5. Typed SQL DSL solutions**

sqlpp11 [9] is particularly interesting because it stands between the traditional runtime query builder and the more strongly static design. Libraries of this family use templates and types to make part of the SQL structure visible to the compiler. This is a significant step beyond purely runtime builders, because part of the structural error space becomes visible earlier.

At the same time, however, a typed SQL DSL usually remains deeply tied to the relational model. Even when very strong as a C++ API, it typically assumes a world in which tables, columns, joins, and aggregates remain the central metaphor. That makes such solutions extremely valuable for SQL, but harder to generalise toward document, graph, or key-value systems.

This is exactly one of the main distinctions relative to the present work. The goal here is not merely compile-time structuring of SQL, but a compile-time logical layer that can be materialised differently depending on the backend family.

## **2.6. Classical ORM frameworks and code generation**

ODB [1] is a characteristic example of an ORM framework that provides rich mapping between classes and relational tables, but depends on code generation and an external toolchain. This makes the library convenient to consume, yet moves complexity into the build process and generated artefacts. A strong dependency appears between the data model, the additional tooling step, and the concrete relational backends.

A further tension is often visible in classical ORM frameworks: the higher the convenience at the application level, the less transparent the path toward the actual query and the performance characteristics of the system may become. This is not necessarily fatal, but it is an important limitation in the academic context of the present thesis, because it makes the relationship between abstraction and materialisation harder to inspect.

## **2.7. Limits of relationally centred abstractions**

For applications that must work with document, graph, or key-value systems, the classical ORM model reveals natural limits. Some concepts — such as JOIN, foreign keys, and rigid tabular schemas — do not have direct equivalents in every storage model. Generalising ORM beyond the relational world therefore cannot be achieved merely by generating SQL.

This observation matters not only for classical ORM frameworks, but also for native APIs and typed SQL DSLs. Even when technically strong, they often assume that SQL remains the central language of the data world. The present work begins from a different premise: the logical model should be primary, while relational materialisation is only one possible interpretation.

**Table 2. Limitations of the different solution families relative to the multi-backend objective**

| <b>Solution family</b> | <b>What it does well</b>                               | <b>Where the limitation emerges</b>                                                 |
|------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------|
| Native API             | maximal control<br>and driver-level<br>closeness       | no shared logical model and large duplication<br>once more than one backend is used |
| Runtime builders       | better readability<br>and query<br>encapsulation       | validation remains mostly runtime and often<br>SQL-centred                          |
| Typed SQL DSL          | stronger static<br>structure for SQL<br>queries        | the logic usually remains confined to the<br>relational metaphor                    |
| Classical ORM          | convenient object<br>mapping for<br>relational systems | dependence on code generation and difficult<br>transfer to non-relational models    |

## 2.8. Positioning of the present work

The proposed library is positioned as a Compile-time Object-Relational Mapping solution that combines strong static typing, absence of external code generation, and a connector-driven model for transferring one logical abstraction across different backend families. Compared to the native API approach, it reduces duplication of logic and the risk of late-discovered structural errors. Compared to runtime query builders, it shifts structural validation into the compiler. Compared to classical ORM frameworks, it avoids external generators and puts explicit emphasis on the connector contract.

The key distinction, however, is deeper. The present work does not treat compile-time techniques merely as a way to build a “smarter SQL builder”. It treats them as a way to formalise the logical model of data access itself. That is precisely what allows the following chapters to move from the theory of typed models toward architecture, connector contracts, and real multi-backend scenarios.



## **2.9. Conclusions from the comparative analysis**

The comparison with existing solutions leads to three principal conclusions. First, the native-API approach remains indispensable as a baseline for control and performance, but does not provide a satisfactory level of logical abstraction. Second, runtime query builders and typed SQL DSL solutions improve work with SQL, but do not fully solve the problem of a backend-independent logical layer. Third, classical ORM frameworks offer high abstraction, but often at the price of tooling complexity and limited suitability outside the relational world.

It is exactly this positioning that justifies the more detailed discussion in the next chapter of two separate but connected questions: which language and theoretical mechanisms make a compile-time ORM approach possible, and how one and the same logical model can be interpreted across relational and non-relational systems.

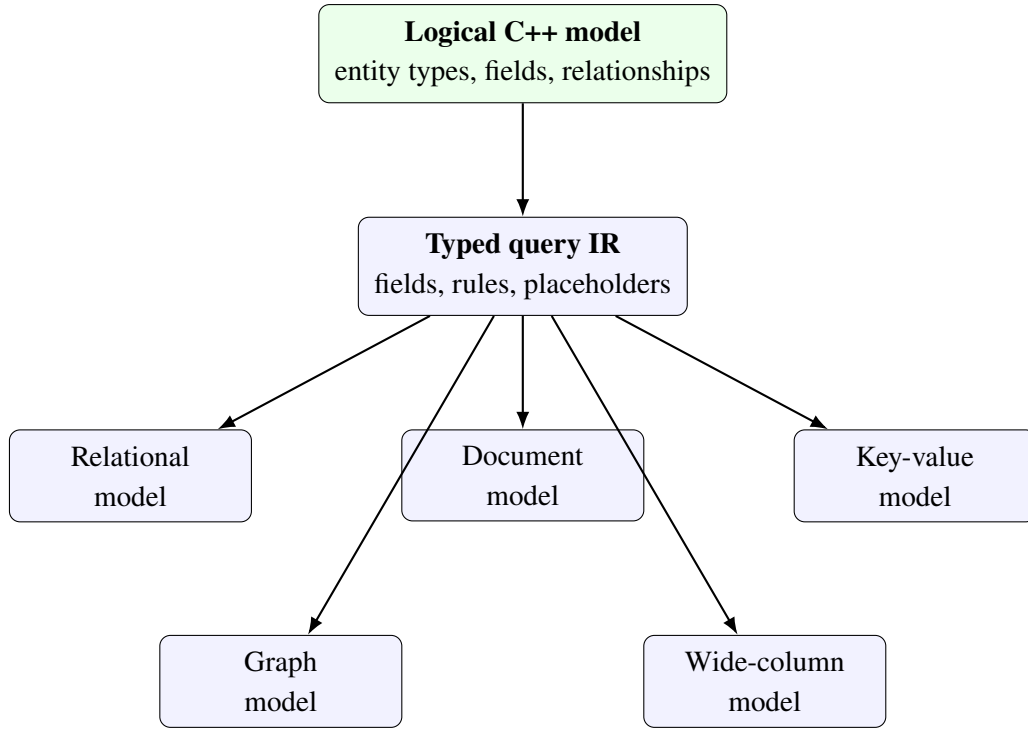
### **III. THEORETICAL FOUNDATIONS OF COMPILE-TIME OBJECT-RELATIONAL MAPPING AND STORAGE MODELS**

The theoretical basis of the present work is not reducible to a single language feature or a single database type. The library combines ideas from ORM theory, static typing in C++, compile-time construction of intermediate representations, and the comparative analysis of different storage models. To understand the architecture developed later, it is necessary first to separate the logical layer of data description from the physical layer of backend-specific materialisation.

Within this thesis, the term Compile-time Object-Relational Mapping should not be interpreted narrowly as a SQL-only technique. More precisely, it denotes an architectural approach in which the data model, the query form, and part of the admissibility constraints are described through the type system of the language rather than through runtime configuration. This makes it possible to carry one and the same logical structure toward relational, document, key-value, graph, and wide-column systems without hiding or falsifying backend-specific differences.

#### **3.1. Taxonomy of the problem domain**

Before the individual components are discussed, it is useful to fix the general picture of the problem domain. On one side stands the C++ model, in which the application describes entity types, fields, relationships, and constraints. On the other side stand different families of storage systems, each with its own model of physical organisation, access, and optimisation. Between them lies the typed query IR, which serves as the mediator between application logic and the concrete connector layer.



**Figure 2. Taxonomy of the logical model, query IR, and backend families**

Figure 2 highlights the central theoretical assumption of the library: what backends share is not their syntax, but the logical model of data and operations. A valid abstraction must therefore be positioned above SQL, BSON, Redis commands, Cypher, or CQL. It must operate at the level of fields, predicates, projections, relationships, and admissible operation classes.

### 3.2. Compile-time Object-Relational Mapping as an architectural approach

Classical ORM connects the object model of an application with its persistent schema [2]. In the compile-time variant, that connection is built not through runtime reflection or configuration files, but through types, templates, and `constexpr` values. In practice this means that field selection, admissible operations, and part of backend compatibility are checked by the compiler. Errors such as references to non-existent fields or attempts to use `JOIN` on a connector that lacks `supports_joins` cease to be runtime surprises.

An essential point is that compile-time ORM does not mean the absence of runtime data. Parameter values are still supplied at execution time, but the logical query form has already been fixed and validated. This yields a clear distinction between *logical structure* and *runtime values*. That distinction is fundamental for the thesis because it makes it possible to combine

static guarantees with real interaction against external databases.

From a theoretical perspective, this approach brings the ORM layer closer to a compiler pipeline. Application code describes a semantic structure, compile-time components turn that structure into an intermediate representation, and only the lowest layer materialises it into a backend-specific language or protocol. For that reason, the later architectural decisions in the library can be analysed through notions such as intermediate representation, static checking, and backend materialisation.

### 3.3. Static data modelling in C++

In the library, the domain model is expressed through C++ aggregates whose fields are declared using `property<T,Name>`. Each field thus carries both a value type and a symbolic field name. In addition, `table_name_trait<T>` links the type to a logical storage container — a table, collection, key prefix, label, or some other backend-specific notion.

This organisation has an important theoretical consequence: the C++ type becomes the primary carrier of the logical schema. Backends do not impose a separate user-level model; they merely interpret the same model differently. This is precisely why the same logical object may later become a relational table, a document with nested or referenced fields, a key-value structure, or a graph node.

**Table 3. Mapping between C++ constructs and theoretical ORM concepts**

| Construct                              | Theoretical role                 | Practical effect in the library                                |
|----------------------------------------|----------------------------------|----------------------------------------------------------------|
| <code>property&lt;T,Name&gt;</code>    | logical-schema attribute         | carries both value type and symbolic materialisation name      |
| <code>relationship&lt;...&gt;</code>   | logical object relationship      | separates the relationship from the physical storage mechanism |
| <code>table_name_trait&lt;T&gt;</code> | logical container                | serves as a table name, collection, key prefix, or label       |
| <code>field&lt;&amp;T::m&gt;</code>    | attribute address in query space | connects the entity layer to the query IR                      |
| <code>connector_trait&lt;DB&gt;</code> | backend materialisation          | defines how the logical model is executed physically           |

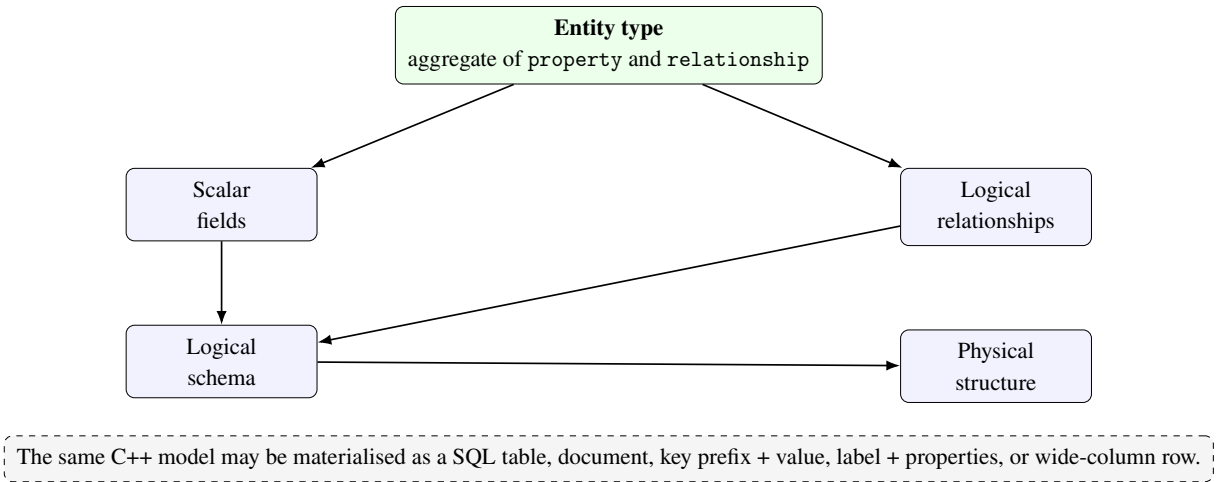
The ORM layer therefore does not begin from a driver or from SQL rendering. It

begins from a static description of the meaning of the data. That is also why the entity layer is foundational for the entire architecture: if this layer is unclear or incomplete, no correct and extensible materialisation can emerge below it.

```
1 template <typename T, detail::string_literal Name>
2 struct property
3 {
4     using value_type = T;
5
6     [[nodiscard]] static constexpr std::string_view column_name() noexcept
7     {
8         return Name.view();
9     }
10
11     T value{};
12 };
13
14 enum class store_as { reference, embed };
15
16 template <store_as Strategy, typename T, detail::string_literal Name>
17 struct relationship
18 {
19     static constexpr store_as strategy = Strategy;
20     using related_type = T;
21
22     [[nodiscard]] static constexpr std::string_view field_name() noexcept
23     {
24         return Name.view();
25     }
26 };
```

**Listing 1:** Excerpt from property and relationship in the entity layer

Listing 1 shows that the entity layer is minimalistic in form, yet semantically rich. property is not merely a value container; it also carries a name, and relationship is not merely a member field; it fixes the storage strategy as part of the type.



**Figure 3. From C++ entity description to logical schema and physical materialisation**

### 3.4. Typed query IR

Instead of describing a query directly as SQL or another textual representation, the library uses a typed query IR. Its building blocks are `field`, `Rule`, `Placeholder`, and query objects such as `select_query`, `insert_query`, `update_query`, and `delete_query`. This model offers two advantages. First, the query structure can be analysed at compile time and participate in `static_assert` checks. Second, the same IR can be rendered into different backend languages and protocols.

The IR therefore plays the role of an intermediate language between the domain model and the execution layer. The user works at a high level of abstraction, while the backend receives a specialised form aligned with its own storage model. This is why the query IR is logically closer to operation semantics than to the syntax of any concrete query language.

```

1  template <typename Response, typename Joins, typename Wheres,
2         typename Limits, typename Groups, typename Orders>
3  struct select_query : select_query_tag
4  {
5      using response_type = Response;
6      using row_type = projected_type<Response>;
7
8      template <typename RuleType>
9          requires is_rule<RuleType>
10     [[nodiscard]] constexpr auto where(RuleType r) const
11     {
12         using NewWheres = detail::append_type_t<Wheres, RuleType>;
13         return select_query<Response, Joins, NewWheres, Limits, Groups, Orders>(
14             response_, joins_,
15             detail::tuple_cat(wheres_, detail::orm_tuple<RuleType>(r)),
16             limits_, groups_, orders_);

```

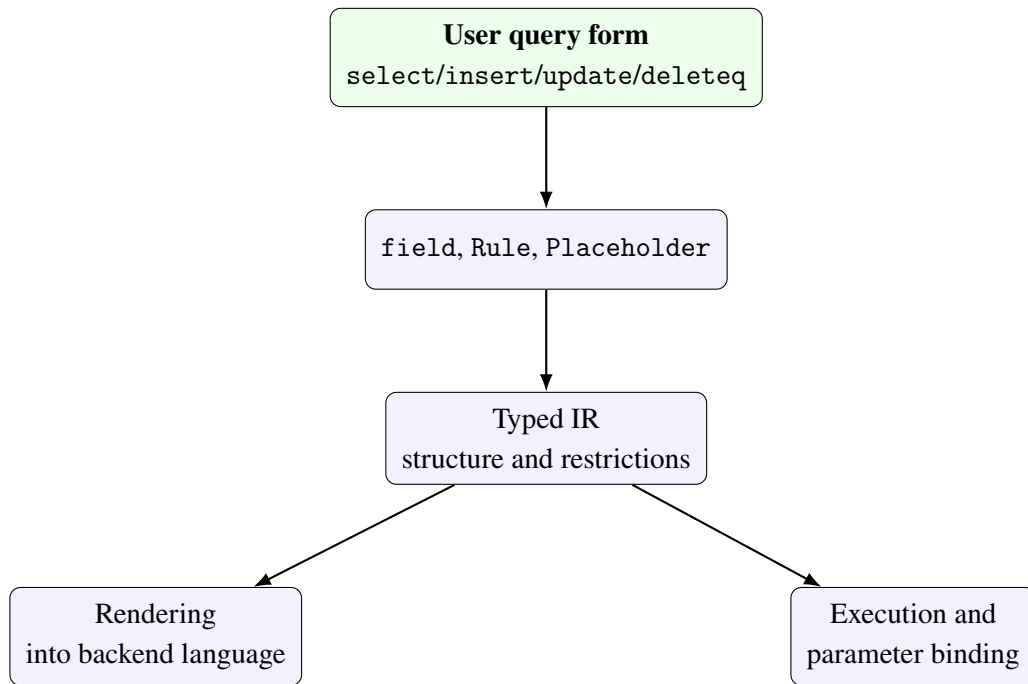
```

17     }
18 };

```

**Listing 2:** Excerpt from `select_query` and `where` clause composition

Listing 2 illustrates an important theoretical property: the query builder does not mutate text, but constructs new typed objects. This means that a sequence such as `select(...).where(...).group_by(...)` can be interpreted as the construction of a new type-level structure rather than as a series of string mutations.



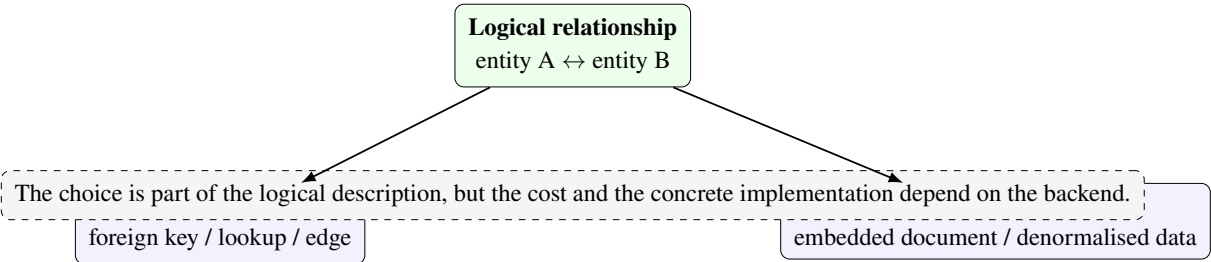
**Figure 4.** Typed query-IR pipeline from application API to backend execution

### 3.5. Relationship theory and storage strategies

Relationships between objects are among the hardest aspects of extending ORM beyond the relational world. In a relational context, a relationship is often materialised through a foreign key and a JOIN. In a document context, however, the same logical relationship may be realised as an embedded document, while in a key-value context it may require secondary lookup or denormalisation.

The library abstracts these options through `relationship<Strategy,T,Name>` and `store_as::reference/store_as::embed`. `reference` denotes a logical external reference, which the backend may realise through a foreign key, lookup, separate key, or graph edge. `embed` denotes inclusion of the related data in the same physical structure. The relationship is thus

described once at the logical level, while physical interpretation remains the connector’s task.



**Figure 5. The two principal relationship materialisation strategies**

**Table 4. Comparison between reference and embed strategies**

| Strategy  | Logical idea                               | Natural backend families               | Primary trade-off                                      |
|-----------|--------------------------------------------|----------------------------------------|--------------------------------------------------------|
| reference | separate identity of the related object    | SQL, graph systems, key-based lookup   | a separate navigation or joining mechanism is required |
| embed     | physical inclusion in the parent structure | document systems, denormalised records | independent update and reuse become harder             |

### 3.6. Relational model

The relational model organises data into tables, rows, and columns. Its main strengths are a clear schema, a declarative query language, and formally defined integrity constraints. For ORM systems this is the natural environment: the properties of a C++ object map easily to columns, and relationships map naturally to foreign keys. Operations such as JOIN, GROUPBY, and transactions belong to the normal language model.

The relational backend is therefore the natural baseline scenario against which one can judge how far an ORM library preserves its usefulness outside the SQL world. In the present work, that baseline later materialises primarily through SQLiteDB, while portability of the IR is further analysed through PostgreSQLDB and MySQLDB.



### 3.7. Document model

The document model organises data into collections of documents whose fields may be nested and whose physical shape need not be identical across all records. This makes it naturally compatible with the embed strategy and more flexible for denormalised structures. On the other hand, directly projecting relational abstractions onto a document backend can be misleading. Not every relational operation has an equivalent with the same semantics or cost.

Within the present work, the document model is important as a test of whether the query IR expresses logic rather than the syntax of a particular language. If one and the same IR can be interpreted as a BSON-like filter and projection, without the user writing native document queries, then a genuinely storage-agnostic layer of abstraction has been achieved.

### 3.8. Key-value model

Key-value systems organise access around a key and its associated value. They are especially effective when the access pattern is explicit and centred around identifier-based lookup. Their disadvantage is that general predicate-based queries and relational operations are strongly limited. An ORM abstraction over a key-value backend therefore cannot promise the same expressive power as over a SQL system.

This model is useful for distinguishing between the *logical object* and the *admissible access pattern*. The object may remain the same, but not every query is meaningful against every physical organisation. In the repository, this becomes visible later through RedisDB, where lookup logic is natural, while join-oriented behaviour is architecturally inappropriate.

### 3.9. Graph model

The graph model represents data as nodes, edges, and properties. Its strength lies in expressing relationships as first-class objects rather than as values derived from foreign keys. This makes it suitable for scenarios involving rich navigation and traversal. At the same time, the graph model imposes its own query language and its own intuition for selection and filtering.

For the library, this is a revealing scenario because it demonstrates that `connector_trait` can transfer a shared query IR toward MATCH/RETURN/WHERE structures without forcing the user to write Cypher directly. If that is possible, then the common layer is indeed positioned above the concrete syntax layer.

### 3.10. Wide-column model

Wide-column systems such as Cassandra are based on distributed tables organised around partition and clustering keys. Although their syntax may resemble SQL, their access theory is different: a query is correct only if it follows the physical key model of data distribution. The mere existence of a field in the C++ model does not imply that it may be used freely in WHERE clauses.

This is exactly the type of limitation that justifies the library’s capability- and constraint-based design. It is not enough for a query IR to be type-correct; it must also be admissible with respect to the storage model. For that reason, the wide-column model is especially valuable for the theoretical defence of compile-time constraints.

**Table 5. Comparison of major storage models**

| Model       | Storage unit    | Typical relationships         | Primary constraint                                          |
|-------------|-----------------|-------------------------------|-------------------------------------------------------------|
| Relational  | table/row       | foreign key, join             | consistent schema discipline and strong typing expectations |
| Document    | document        | embed or reference            | not every relational operation is natural or inexpensive    |
| Key-value   | key and value   | key-based lookup, hash fields | predicate queries outside the key path are strongly limited |
| Graph       | node and edge   | traversal relationships       | specialised query language and graph semantics              |
| Wide-column | partitioned row | key-oriented dependencies     | queries must follow partition design rules                  |

### 3.11. Test evidence behind the theoretical claims

The theoretical claims of this chapter are not detached from the code. The entity layer is validated directly in `tests/unit/test_property.cpp` and `tests/unit/test_relationship.cpp`, where column naming, entity recognition, the distinction between `store_as::reference` and `store_as::embed`, and one-to-many inference are checked. The typed query IR is covered in `tests/unit/test_query.cpp`, which demonstrates field-based predicates, join forms, update composition, and delete semantics. In addition, `tests/unit/test_cpp26_re`

`flection.cpp` shows that the compile-time description of fields can also be extended through a reflection-aware path when the toolchain supports it.

This relation between theory and test evidence is crucial. It shows that the presented notions are not free academic metaphors, but real architectural units that can be inspected in the source code and validated through automated tests.

### **3.12. Boundaries of the shared abstraction**

An important conclusion follows from the above. A unified ORM abstraction does not mean that all backends become equivalent. Unification is possible at the level of logical model, field, relationship, query IR, and basic operation categories. Beyond that point, backend differences reappear: SQL JOIN has no universal meaning in Redis; embedding has no identical cost in a classical SQL table; graph traversal is not reducible to an ordinary SELECT; and the partition-driven restrictions of Cassandra cannot be masked as ordinary predicate rules.

For that reason, the architecturally correct strategy is not to hide all differences, but to make them explicit and formally controlled. That is precisely the role of `connector_trait`, capability tags, and compile-time restrictions, which are examined in the following chapters. There it becomes visible how the theoretical framework of the present chapter is turned into a working library architecture.

## IV. LIBRARY ARCHITECTURE

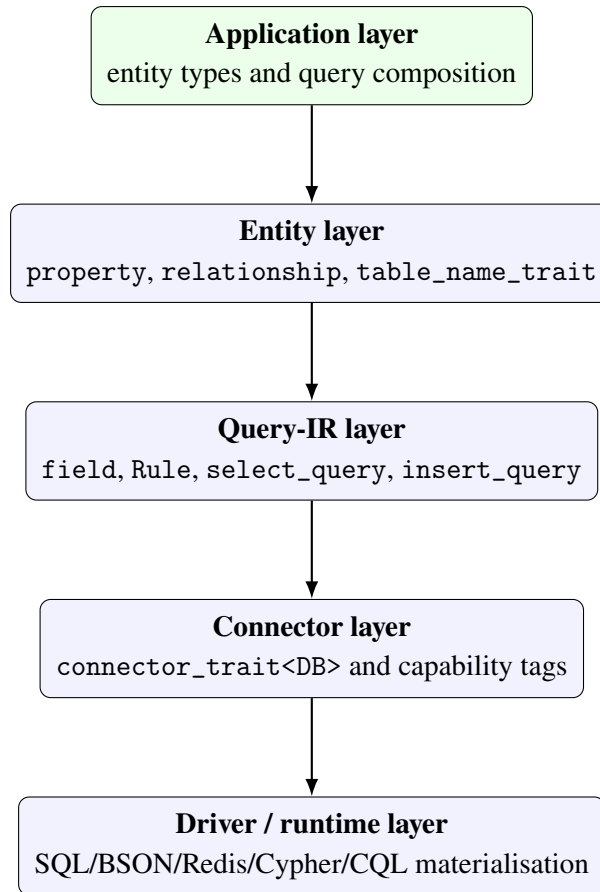
The architecture of the library is organised around a clear separation between the logical data model, the intermediate representation of queries, and the backend-specific execution layer. That separation is central to the goal of the project: changing the database should require minimal change in application code and, ideally, only a change in the connector type. To achieve this, the architecture is built from several layers, each with its own responsibility and with strictly delimited interfaces.

The theoretical chapter showed that unification is possible at the level of the logical model and the query IR, but not by ignoring the differences between backend families. The present chapter takes the next step: it shows how that theoretical position is turned into a concrete architecture. The library is therefore treated not merely as an idea, but as an organised system of types, contracts, and execution paths.

### 4.1. Architectural goals and constraints

The architecture was designed under several constraints acting simultaneously. First, the user-facing API must remain within idiomatic C++, without introducing a separate DSL and without relying on runtime reflection. Second, the query structure must be representable as a `constexpr` object so that a significant portion of validation can be performed by the compiler. Third, the backend layer must remain extensible without inheritance from a common virtual interface, because such an interface would hide important backend differences and move correctness checks toward runtime.

A fourth constraint follows from the multi-backend nature of the system itself. The library must support SQL and non-SQL connectors without false equivalence. This means that the architecture must simultaneously provide a shared API and express the limitations of a concrete backend in a formal and testable manner. The choice of a trait-based connector layer together with a capability mechanism follows naturally from this requirement.



**Figure 6. Layered architecture of the library**

Figure 6 shows that the library follows a consistent vertical organisation. Application code does not communicate directly with the driver; it works with C++ types and query builders. The driver, in turn, does not know domain semantics, but only the materialised form generated by the connector layer. This is precisely why the architecture can remain both strict and extensible.

## 4.2. Layered organisation and responsibility separation

The topmost layer is the application code, which defines entity types, constructs queries through `select`, `insert`, `update`, and `deleteq`, and submits them to `db<DB>`. The next layer is the logical model of the library — the entity descriptions and the query IR. The third layer is the connector abstraction, where `connector_trait<DB>` materialises the same logic into backend-specific syntax and behaviour. At the bottom stands the concrete driver, wire protocol, or mock implementation.

This architecture has an important practical effect. If the user replaces `SQLiteDB` with `MongoDB`, the query IR remains logically the same, but the connector interprets it as filter/projection execution rather than SQL rendering. This does not mean that every IR is admissible on every

backend; it means that the *form of abstraction* is shared, while admissibility is determined formally through the capability mechanism.

**Table 6. Principal architectural layers and their responsibilities**

| Layer          | Main artefacts                                 | Responsibility                                                         |
|----------------|------------------------------------------------|------------------------------------------------------------------------|
| Application    | entity types,<br>query-builder calls           | describes domain logic and formulates operations<br>in idiomatic C++   |
| Logical        | property, relationship, field, Rule            | models data and query structure independently of a<br>concrete backend |
| Connector      | connector_trait<DB>,<br>capability tags        | checks admissibility and materialises the IR into<br>backend behaviour |
| Runtime/driver | concrete connection<br>handles and driver APIs | executes the materialised query and returns results                    |

Table 6 shows that the library does not rely on a single monolithic object that does everything. Instead, decisions are localised in separate layers. This matters both for extensibility and for academic analysis, because each claim can be traced to concrete types and modules.

### 4.3. The entity layer as architectural foundation

The entity layer is built around `property<T,Name>`, `relationship<...>`, and `table_name_trait<T>`. It plays a dual role. On the one hand, it represents the C++ domain model. On the other hand, it serves as a logical description of the persistent schema or structure. This means that the architecture does not construct a separate runtime metadata registry. Instead, the type system contains enough information to derive field names, value types, relationships, and the target storage container.

Especially important is the fact that the entity layer is independent of the database family. Neither `property` nor `relationship` is SQL-specific. They describe logical attributes and logical relations. This creates the foundation for comparable behaviour across relational and non-relational backends. At the architectural level, this is exactly what makes it possible for the thesis to argue that the C++ model is the primary carrier of the logical schema.

## 4.4. Query IR as the central architectural component

The query IR is the central architectural component. It combines field references, predicates, placeholders, and additional clauses such as `order_by`, `group_by`, and `limit`. Instead of working with textual queries, the library constructs typed objects whose template parameters encode the logical structure of the operation.

From the architectural point of view, this has several consequences. First, structural validation occurs early. Second, the query IR can be analysed by the connector layer without losing semantic information. Third, prepared queries can preserve not an SQL string, but the intermediate representation itself, which is more natural for a library oriented around the C++ type system.

Fourth, the query IR is the single zone in which entity description and later backend materialisation meet. In that sense it is the architectural hinge of the whole library: high enough to remain storage-agnostic, yet concrete enough to be executed.

## 4.5. The connector layer and the formal contract

Formally, the connector layer is implemented through specialisation of `connector_trait<DB>`. This approach is analogous to standard trait templates such as `std::iterator_traits`. The type `DB` may be a simple wrapper over a connection handle, while the entire ORM logic remains in the trait specialisation. In this way, the library does not require the user or the connector author to inherit from a common base class.

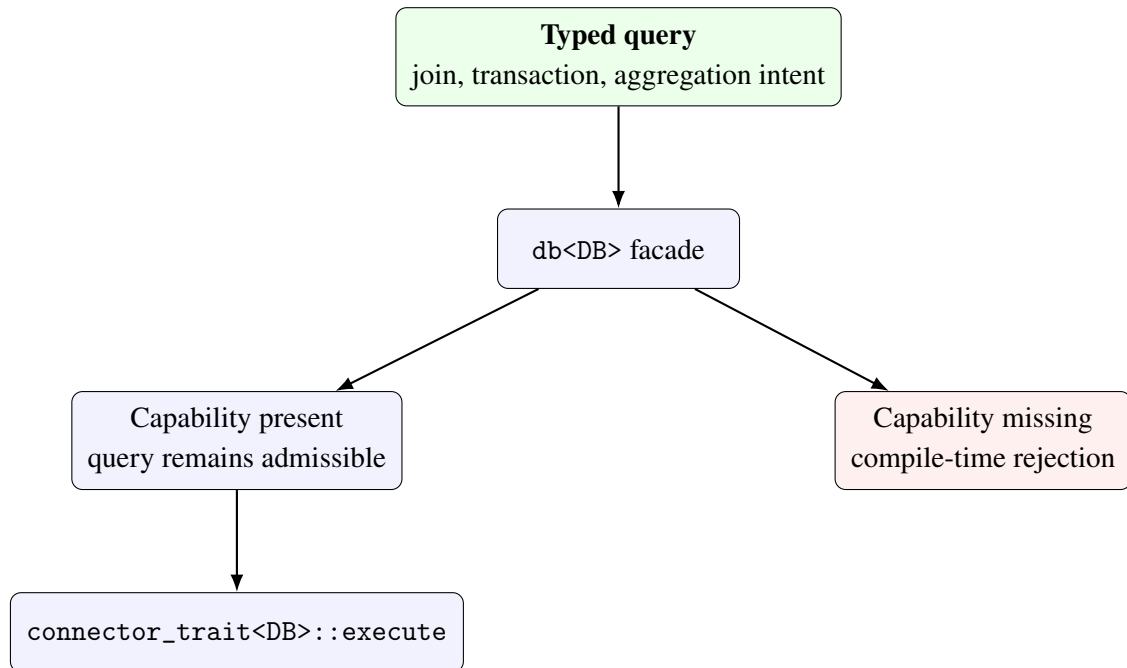
`connector_trait<DB>` defines at least three central aspects: `wire_type`, `cursor_type`, and `execute(...)`. The specialisation may also declare capability tags such as `supports_joins`, `supports_transactions`, `supports_aggregation`, or `supports_concurrent_execute`. The presence or absence of these pseudo-markers is sufficient for the upper layers to impose compile-time restrictions.

```
1  template <typename DB>
2  concept is_connector = requires {
3      typename connector_trait<DB>::template wire_type<int>::type;
4      typename connector_trait<DB>::cursor_type;
5  };
6
7  template <typename DB, typename Cap>
8  inline constexpr bool has_capability =
9      detail::capability_check<DB, Cap>::value;
```

---

**Listing 3:** Excerpt from the formal connector contract

Listing 3 shows that the contract is structural rather than inheritance-based. This is a decisive architectural choice. If a virtual interface existed with runtime dispatch, some of the type-checkable information would be lost. The library instead prefers a compile-time structure in which the requirements remain visible both to the compiler and to the connector author.



**Figure 7. Capability gating between the query IR and the connector layer**

Figure 7 is important because it shows where the architecture decides admissibility. This is not the driver and not a runtime log. The decision is taken in the public API layer, which already has access both to the query IR and to the trait-level information about the connector.



**Table 7. Role of the principal capability tags in the architecture**

| Capability                         | Architectural meaning                                                                               |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>supports_joins</code>        | admits join-based query forms for the selected backend                                              |
| <code>supports_transactions</code> | allows use of <code>transaction_guard</code> together with <code>begin/commit/rollback</code> hooks |
| <code>supports_aggregation</code>  | marks support for aggregation operations in the logical query layer                                 |
| <code>supports_embedding</code>    | makes native support for embedded structures explicit                                               |
| <code>supports_upsert</code>       | marks backends for which upsert belongs to the natural execution contract                           |
| <code>supports_bulk_insert</code>  | denotes the existence of a more efficient bulk-write path                                           |

#### 4.6. The user-facing `db<DB>` facade

The class `db<DB>` is the public facade layer. It encapsulates a reference to a concrete backend object and provides a unified execution API. In practice, this layer has two roles. The first is convenience for the user: the library is used through the same interface regardless of backend. The second is architectural filtering: this is precisely where compile-time checks for capabilities and backend compatibility are performed.

Consequently, `db<DB>` is not merely a thin wrapper over `execute`. It is the place where the library formalises the boundary between admissible and inadmissible behaviour for the selected backend. It is exactly this layer that enables error messages such as “this connector does not support JOIN operations” instead of allowing such errors to surface later as ambiguous runtime behaviour.

```

1  template <typename DB>
2      requires is_connector<DB>
3  class db
4  {
5  public:
6      template <typename Query>
7      auto operator<<(Query q)
8      {
9          if constexpr (is_select_query<Query>)
10         {
11             if constexpr (decltype(q.join_clauses())::size > 0)
12             {
13                 static_assert(has_capability<DB, cap::supports_joins>,

```

```

14         "orm::db: this connector does not support JOIN operations.");
15     }
16 }
17 return connector_trait<DB>::execute(*conn_, std::move(q));
18 }
19 };

```

**Listing 4:** Excerpt from the db<DB> facade layer

Listing 4 shows why db<DB> is more than a convenient facade. It is the formal location where query structure and connector capability information meet. This is where the compile-time architecture becomes a real limitation on API use.

## 4.7. Migrations as an architectural extension

The architecture also includes a prototype migration layer through migrate<DB>, live\_schema, entity\_meta, and DDL operations such as create\_table\_op, add\_column\_op, drop\_column\_op, and alter\_column\_type\_op. This layer is particularly important from the standpoint of the thesis because it makes explicit the relation between the C++ model and the real storage schema.

The migration layer is not uniformly complete for every backend, but architecturally it demonstrates that the library is not limited to query execution. With a sufficiently rich connector\_trait<DB> specialisation, entity description can also participate in schema evolution. This changes the very meaning of the ORM layer: it becomes not only an interface to data, but a potential source of schema-aware tooling.

## 4.8. Architectural invariants

On the basis of the analysed layers, several invariants can be formulated. First, application code should never depend directly on driver APIs. Second, query logic is formulated in a typed IR rather than in backend-native strings. Third, extension toward a new backend proceeds through trait specialisation and capability declarations rather than through changes to the query API itself. Fourth, if an operation is inadmissible for a backend, this should become visible as early as possible — ideally at compile time.

These invariants make the architecture both engineering-sound and academically defensible. They allow every future extension to be judged not only by whether it “works”, but by whether it preserves the already formulated layers and boundaries.

## **4.9. Architectural conclusion**

In summary, the architecture of the library separates the problem into three levels: logical model, typed query IR, and backend materialisation. This separation enables strict type discipline, formal extensibility, and controlled behaviour across distinct storage models. The next chapter examines this architectural decision more concretely through the implementation of the core subsystems, where it becomes clear how the architectural plan is turned into executable C++ code.

## V. IMPLEMENTATION OF THE CORE SUBSYSTEMS

Once the architectural model has been defined, the next question is how it is realised in terms of concrete C++ constructs and how the individual subsystems interact. This chapter examines the principal building blocks of the library and follows them from entity descriptions to query execution, prepared queries, migrations, and the enforcement of constraints.

The focus here is no longer on layers as an abstract architectural scheme, but on their actual mechanics. How exactly are where clauses accumulated? How is a query object preserved for repeated use? How does `migrate<DB>` transform the difference between the entity model and `live_schema` into DDL operations? How does the thread-safety layer guarantee correct use of connections? The answers to these questions show how far the library is developed as a real engineering system.

### 5.1. Repository-backed mini case study: User and Post

To keep the discussion anchored in the real repository, this chapter uses as its main mini case study the User and Post types from `tests/integration/test_mockdb.cpp`. They are especially useful because they combine scalar fields, table naming, and a join scenario. The same example is later reused for prepared queries, result typing, and SQL rendering assertions.

```
1 struct Post
2 {
3     orm::property<int, "id">          id;
4     orm::property<int, "author_id">    author_id;
5     orm::property<std::u8string, "body"> body;
6 };
7
8 struct User
9 {
10    orm::property<int, "id">          id;
11    orm::property<std::u8string, "name"> name;
12    orm::property<double, "score">    score;
13 };
```

**Listing 5:** Entity types from `tests/integration/test_mockdb.cpp`

Listing 5 shows not only how the entity model is described, but also how it participates directly in the tests. `table_name_trait<User>` and `table_name_trait<Post>` fix the logical containers "users" and "posts", while member pointers such as `&User::id` and `&Post::author_id` later appear in the query builder. This is a good illustration of one of the central

themes of the thesis: the model, the query form, and the execution tests all rely on the same compile-time artefacts.

## 5.2. Scalar properties and naming of storage containers

The implementation of `property<T,Name>` binds a value type to a compile-time name. The `column_name()` function provides unambiguous access to the symbolic field name and is used by connectors when rendering toward SQL columns, document fields, or other backend-specific notions. This construction is both minimalistic and sufficient because it does not introduce an external metadata registry.

Naming of the target storage container is realised through `table_name_trait<T>`. Although the name is inherited from ORM tradition, architecturally it plays a more general role: in SQLite and PostgreSQL it denotes a table, in MongoDB a collection, in Redis a key prefix, and in Neo4j a label. In this way, the library uses a unified logical interface for addressing physically distinct structures.

In practical terms, this means that a connector author does not extract information from an external schema or from a runtime registry, but from the type system itself. This reduces boilerplate and makes naming errors easier to localise during compilation or in unit tests.

## 5.3. Field wrappers, rules, and placeholders

A field in a query expression is represented by `field<&T::m>`. This is a `const` `expr` wrapper over a member pointer that carries enough information about the table type, the value type, and the column name. Operators over `field` construct typed `Rule` objects. Expressions such as `field<&User::id>==Placeholder<int>\{\}` are therefore not strings, but compile-time descriptions of predicates.

The placeholder mechanism has two forms. `Placeholder<T>` models anonymous parameters that are consumed positionally from `execute()` arguments. `orm::ph<T,std::placeholders::_N>` models indexed parameters that allow a single runtime value to be reused in more than one position. This is especially important for backends such as SQLite and PostgreSQL, where indexed placeholders are naturally supported.

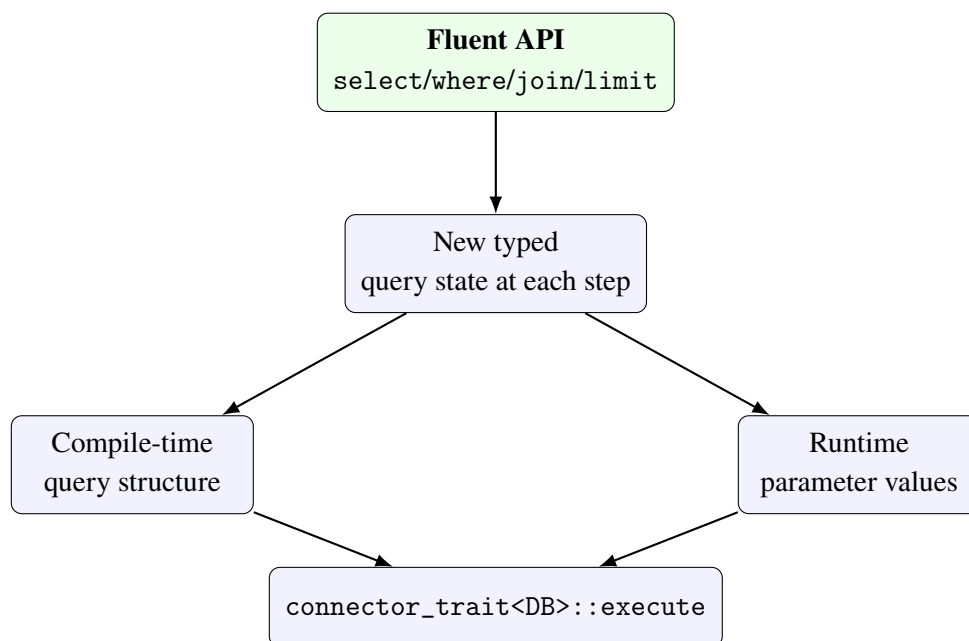
The tests `IndexedPlaceholderRendersQuestionMarkN`, `IndexedPlaceholderReusedSameArgInSQL`, and `TwoDistinctIndexedPlaceholdersRenderCorrectSlots` in `tests/integration/test_mockdb.cpp` show that this subsystem is not merely theoretical

ornamentation, but a real mechanism for expressing more precise parameter-binding scenarios.

## 5.4. Construction of query objects

The factory functions `select`, `insert`, `update`, and `deleteq` create query objects onto which `where`, `join`, `order_by`, `group_by`, and `limit` are layered successively. Crucially, each of these operations does not modify a string, but returns a new typed object with extended internal state. As a result, the query builder is fluent in appearance and static in nature.

This allows the tests to check not only the final rendering result, but the very shape of the intermediate representation — for example, the presence of an `INNERJOIN`, the order clause, or the resulting response tuple type. In `tests/integration/test_mockdb.cpp`, this appears in `SelectWithInnerJoin`, `SelectWithOrderByDesc`, `SelectWithLimit`, and many other scenarios that validate different phases of query composition.



**Figure 8. Interaction between query composition, compile-time structure, and runtime parameters**

Figure 8 shows that the implementation operates in two modes simultaneously: the structural form of the query is fixed in the type, while the concrete values are supplied separately. This is exactly what makes it possible to combine compile-time checking with runtime parameterisation.

## 5.5. Execution, results, and field access

At execution time, `db<DB>` forwards the query IR to `connector_trait<DB>::execute`. The result is returned as `orm::result<Row, Fields>`, where `Row` is a tuple type and `fields` preserves information about the selected fields. This design supports both iteration over materialised rows and field access through `get_field<&T::m>`. The implementation therefore remains type-safe even after the query has been executed.

The tests `SelectResultRowTypeIsTuple`, `SelectMultiFieldRowType`, `SelectGetByMemberPointer`, and `SelectGetByIndex` show that the result layer is not merely a container of anonymous rows, but a structured type-safe abstraction. This is especially important for the thesis because it demonstrates that type discipline does not end with the query builder, but continues into result handling.

## 5.6. Prepared queries as a reusable execution artefact

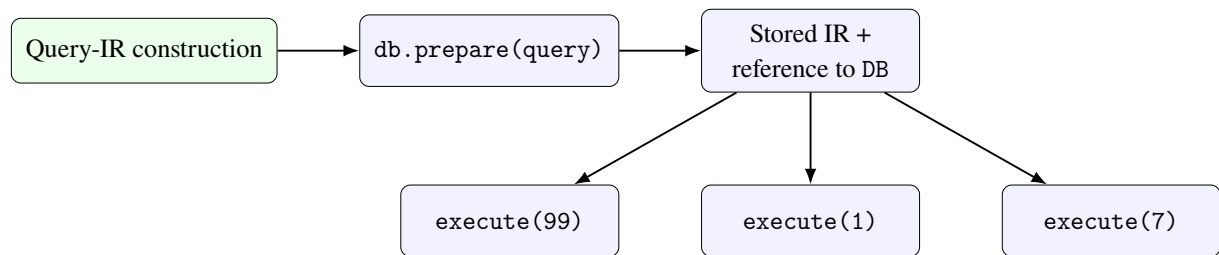
`prepared_query<DB, Query>` extends the execution model. Instead of caching an SQL string, it stores a reference to the backend object together with the query IR itself. This is a natural continuation of the compile-time approach: the library treats the query object as the primary artefact rather than as a temporary step on the way to text.

```
1 template <typename DB, typename Query>
2 class prepared_query
3 {
4 public:
5     prepared_query(DB& conn, Query q) : conn_(conn), query_(std::move(q)) {}
6
7     template <typename... Params>
8     auto execute(Params&&... params) const
9     {
10         return connector_trait<DB>::execute(
11             conn_, query_, std::forward<Params>(params)...);
12     }
13
14     [[nodiscard]] const Query& query() const noexcept { return query_; }
15 };
```

**Listing 6:** Excerpt from `prepared_query<DB, Query>`

Listing 6 reveals an important engineering choice: reuse is organised around the IR rather than around rendered text. This enables one and the same logical query to be executed repeatedly

with different parameter values, without reconstructing the builder form.



**Figure 9. Lifecycle of prepared\_query: one IR, multiple executions**

This behaviour is precisely what is validated by `PrepareReturnsExecutableObject`, `PrepareWithParamExecutesCorrectly`, `PrepareCanBeCalledMultipleTimesWithDifferentParams`, and `PrepareExposesQueryIR` in `tests/integration/test_mockdb.cpp`. These tests prove that the prepared-query object is both executable and inspectable, which is strong evidence of internal consistency in the subsystem.

## 5.7. Migration layer and DDL generation

The implementation of `migrate<DB>` compares the schema described by entity types against `live_schema`. Whenever a difference exists, DDL operations are produced and delegated to `connector_trait<DB>::ddl_for(...)`. This is a good example of clean separation of responsibilities: diff logic is shared, while concrete DDL syntax remains backend-specific.

```

1 [[nodiscard]] std::vector<ddl_op> diff(
2     const live_schema& live,
3     std::span<const entity_meta> entities) const
4 {
5     std::vector<ddl_op> ops;
6
7     for (const auto& entity : entities)
8     {
9         auto live_it = live.find(entity.table_name);
10        if (live_it == live.end())
11        {
12            ops.push_back(create_table_op{entity.table_name, entity.columns});
13            continue;
14        }
15
16        for (const auto& [col, type] : entity.columns)
17        {
18            if (live_it->second.find(col) == live_it->second.end())
19                ops.push_back(add_column_op{entity.table_name, col, type});

```



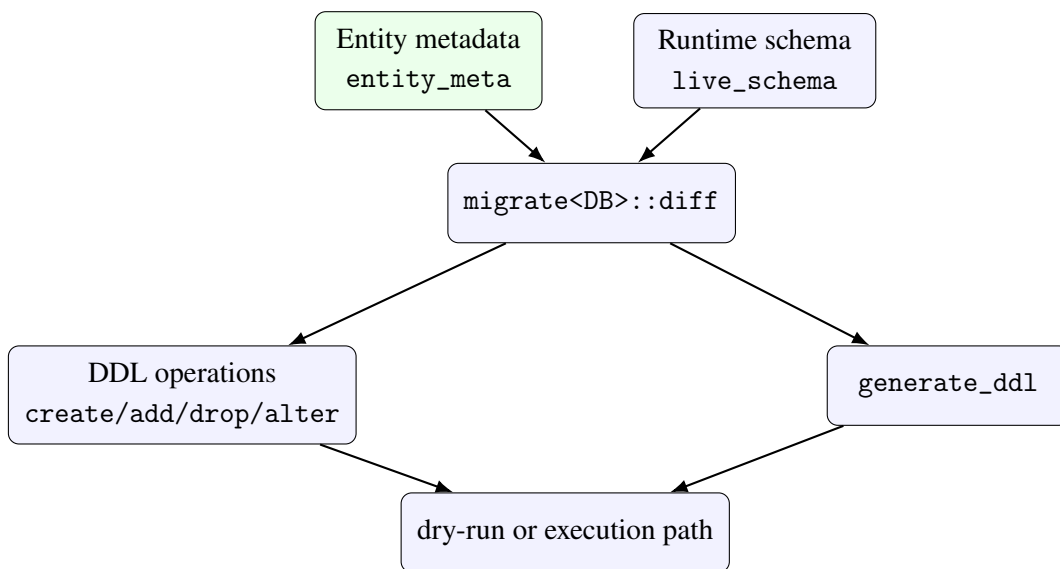
```

20     }
21 }
22 return ops;
23 }

```

**Listing 7:** Excerpt from the migration diff pipeline

It is precisely here that the difference between a *minimal working connector* and a *complete connector* becomes visible. A backend may execute queries without supporting DDL generation. But if it is to participate in schema-aware tooling, its contract must be extended.



**Figure 10.** Flow of the schema-migration subsystem

The tests in `tests/unit/test_schema_migration.cpp` cover table creation, addition and removal of columns, dry-run exit codes, DDL generation, and the state-machine behaviour of `run()`. This is a good example of how higher-responsibility library subsystems are validated through targeted unit tests rather than through integration smoke tests alone.

## 5.8. Thread safety and transaction helpers

The library also contains a dedicated layer for `connection_pool<DB,N>`, `thread_local_db<DB>`, and `transaction_guard<DB>`. These components show that the implementation does not stop at one-off query execution, but addresses more realistic operational scenarios. The important architectural decision is that access to these mechanisms is also capability-based. For example, `connection_pool` requires `supports_concurrent_execute`, while `transaction_guard` requires `supports_transactions`.

```

1  template <typename DB, std::size_t N>
2  class connection_pool
3  {
4      static_assert(
5          requires { typename connector_trait<DB>::supports_concurrent_execute; },
6          "connection_pool requires connector_trait<DB>::supports_concurrent_execute.");
7
8  public:
9      [[nodiscard]] connection_guard<DB> acquire();
10 };
11
12 template <typename DB>
13 class transaction_guard
14 {
15     static_assert(
16         requires { typename connector_trait<DB>::supports_transactions; },
17         "transaction_guard requires connector_trait<DB>::supports_transactions.");
18 };

```

**Listing 8:** Excerpt from the thread-safety helper layer

The compile-time contract therefore does not end at query form, but extends into the lifecycle policies of the connector. This matters for the thesis claim that the library is not merely a query builder, but an infrastructure for systematic and safe backend interaction.

**Table 8. Core subsystems and their test evidence**

| Subsystem         | Main files                                           | Test evidence                                                  |
|-------------------|------------------------------------------------------|----------------------------------------------------------------|
| Entity modelling  | ORM/entity/property.hpp, ORM/entity/relationship.hpp | tests/unit/test_property.cpp, tests/unit/test_relationship.cpp |
| Query composition | ORM/query/select.hpp and related query headers       | tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp   |
| Prepared queries  | ORM/connector/prepared_query.hpp                     | the Prepare* scenarios in tests/integration/test_mockdb.cpp    |
| Schema migration  | ORM/db/migration/migration.hpp                       | tests/unit/test_schema_migration.cpp                           |
| Thread safety     | ThreadSafety/thread_safety.hpp                       | tests/unit/test_thread_safety.cpp                              |

## 5.9. Implementation conclusion

The core subsystems have direct support in the code and in the tests. `tests/unit/test_property.cpp` and `tests/unit/test_relationship.cpp` validate the entity descriptions. `tests/unit/test_query.cpp` demonstrates that the query builder does indeed construct the intended compile-time structures. `tests/integration/test_mockdb.cpp` and `tests/integration/test_sqlite.cpp` validate execution and prepared-query scenarios. `tests/unit/test_schema_migration.cpp` covers the migration diff logic, while `tests/unit/test_thread_safety.cpp` verifies the concurrency layer.

Consequently, the implementation of the core subsystems is not merely a project intention, but an architectural solution supported by systematic verification. This is the most important conclusion of the present chapter: the compile-time ORM architecture in the repository does not remain at the level of abstraction, but materialises into working subsystems with clear responsibilities and explicit test coverage.

## VI. BACKEND SCENARIOS ACROSS RELATIONAL AND NON-RELATIONAL DATABASES

This chapter examines how one and the same C++ abstraction is materialised across different backends. The scenario analysis has two goals. First, it demonstrates that the library is not restricted to a purely SQL interpretation. Second, it clarifies the boundaries of the shared abstraction and the points at which backend-specific constraints must remain explicit.

After the theoretical framework of Chapter III, the question is no longer whether the different storage models are equivalent. Instead, the question is whether one and the same logical ORM form can be adapted to them in a controlled way, without the library beginning to promise operations that a given backend cannot naturally execute. It is precisely here that the value of the capability-based connector contract becomes visible.

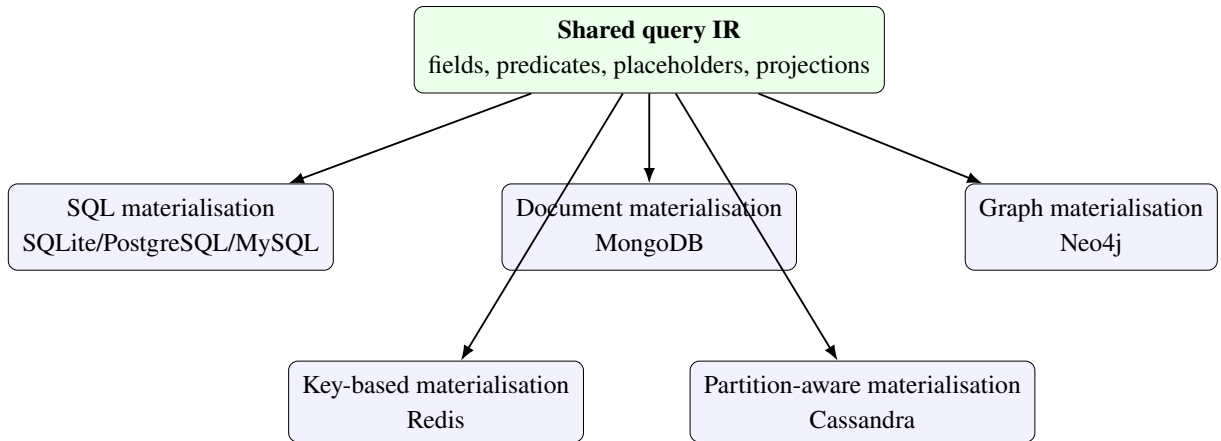
### 6.1. Methodology of the analysis

Each scenario follows the same sequence: theoretical basis, logical mapping from the C++ model, implementation in `connector_trait<DB>`, limitations, and verification through tests. This makes comparison between backends possible. The aim is not to claim that all systems are equivalent, but that one and the same logical ORM abstraction can be adapted in a controlled fashion to distinct storage models.

**Table 9. Common analytical frame for each backend scenario**

| Criterion           | What is evaluated                                                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Theoretical basis   | to which storage model from Chapter III the backend belongs                                                                           |
| Logical mapping     | how <code>property</code> , <code>field</code> , <code>Rule</code> , and <code>table_name_trait</code> are interpreted in the backend |
| Capability profile  | which ORM operations are explicitly admitted and which must be rejected                                                               |
| Execution behaviour | how queries, parameters, and result projections are rendered                                                                          |
| Test evidence       | which unit and integration tests validate the contract in the repository                                                              |

This analytical frame matters because it keeps the comparison honest. SQL backends are not privileged merely because they expose a richer syntax, and NoSQL backends are not judged as “incomplete” merely because they reject operations that are inappropriate for their storage model.



**Figure 11. One shared query IR and multiple backend-specific materialisation paths**

Figure 11 expresses the core claim of the chapter: there are not multiple unrelated ORM libraries in the repository, but one logical library with multiple materialisation paths. This is the strong argument in favour of the connector architecture — backend diversity is managed through a shared IR and an explicit contract rather than by duplicating APIs.

## 6.2. Relational baseline scenario: SQLite

This scenario rests on the relational model discussed in Chapter III. SQLiteDB is the most natural reference backend for the library because it combines real SQL semantics, a compact operational context, and direct integration through the `sqlite3` API. In this backend, property materialises as a column, `table_name_trait` as a table name, and Rule trees as WHERE expressions.

The connector declares `supports_joins`, `supports_transactions`, and `supports_aggregation`, which makes it the closest to classical ORM expectations. It is precisely here that prepared queries, binding logic, `find_one()` behaviour, and result hydration into typed tuples are validated. The integration tests in `tests/integration/test_sqlite.cpp` demonstrate not only correctness of SELECT, INSERT, UPDATE, and DELETE, but also behaviour under a transaction-capable and join-capable connector profile.

From an academic point of view, SQLite plays the role of relational baseline. If a given ORM form is problematic already here, it is unlikely to generalise well toward more exotic backends. For that reason, the present work uses SQLite as a reference point rather than merely as one example among many.

### 6.3. Relational portability: PostgreSQL and MySQL

The theoretical basis here is again the relational model, but the architectural interest is different: to what extent one and the same query IR remains valid across different SQL dialects and different parameter-binding rules. PostgreSQLDB and PostgreSQLLiveDB support `supports_joins`, `supports_transactions`, and `supports_aggregation`, while the tests in `tests/unit/test_postgresql_connector.cpp` and `tests/integration/test_postgresql_live.cpp` show real execution against a live database.

MySQLDB and MySQLLiveDB also declare `supports_joins` and `supports_transactions`, but here the difference in binding model becomes important. While PostgreSQL can naturally address parameters through `\$1`, `\$2`, ..., MySQL relies on positional `?` placeholders. This does not break the shared IR, but it demonstrates that logical portability does not imply byte-for-byte identical backend materialisation.

These SQL family scenarios therefore defend a stronger thesis than “the library works with SQL”: the query IR is general enough for more than one dialect, yet honest enough to leave low-level differences within the connector layer.

### 6.4. Document scenario: MongoDB

This scenario is based on the document model from the theoretical chapter. In MongoDB, `property` is interpreted as a document field, while query predicate trees are translated into BSON-like filters. `select` expressions materialise as projections, and the notion of table in ORM terms becomes a collection. This is precisely where the practical value of a storage-agnostic query IR becomes visible: the user does not describe `\$eq`, `\$and`, or projection directly, yet the connector can derive them from the same logical form.

An important detail is that `connector_trait<MongoDB>` does not declare SQL-style capability tags. This is not an omission, but a correct description of the connector contract. There is no attempt to claim that the Mongo backend is join-equivalent to relational connectors. Instead, it provides a document-native interpretation of the query IR. The unit and integration tests in `tests/unit/test_mongodb_connector.cpp` and `tests/integration/test_mongodb_live.cpp` validate this adaptation against a real driver layer.

The architectural value of this scenario is that it demonstrates an important restriction: a library may be multi-backend without claiming that all backends support the same operations.

It is exactly this sort of controlled partiality that makes the design scientifically defensible.

## 6.5. Key-value scenario: Redis

This scenario rests on key-value theory. In Redis, one and the same entity model can be materialised in two principal ways: as SET/GET for single-field structures or as HSET/HGETALL for multi-field structures. `table_name()` participates in generation of the key prefix, while access is concentrated around primary-key equality.

Here it becomes very clear that the shared query IR does not imply identical expressiveness. RedisDB deliberately does not declare `supports_joins`, `supports_transactions`, or `supports_aggregation`. This restriction is part of the correct contract rather than a weakness of the architecture. The unit and live tests in `tests/unit/test_redis_connector.cpp` and `tests/integration/test_redis_live.cpp` prove that the library correctly materialises key-based access without promising unsupported operations.

This scenario is especially valuable for the thesis because it forces the architecture to distinguish clearly between the *logical data model* and the *admissible access model*. That distinction is easy to forget in SQL-first libraries, but here it is elevated into a central principle.

## 6.6. Graph scenario: Neo4j

The scenario for Neo4j is based on the graph model. The entity type materialises as a label and a set of properties, while the query IR is translated into MATCH, RETURN, and WHERE fragments. This is an important architectural test because graph-query semantics are among the furthest from the classical SQL model. Nevertheless, the logical elements — fields, predicates, filtering, and result projections — remain recognisable.

Neo4jDB and Neo4jLiveDB declare `supports_transactions`, but not SQL join/aggregation capabilities. This is meaningful: graph traversal is not the same as SQL join, even though both connect data at a logical level. The unit and integration tests in `tests/unit/test_neo4j_connector.cpp` and `tests/integration/test_neo4j_live.cpp` show that the abstraction can adapt one and the same logical model toward a graph backend if the connector takes responsibility for correct materialisation.

From the standpoint of the thesis, this is one of the strongest demonstrations that the query IR is semantic rather than syntactic. If the IR were merely under-specified SQL, the Neo4j scenario would collapse. The fact that a working graph materialisation is possible supports the

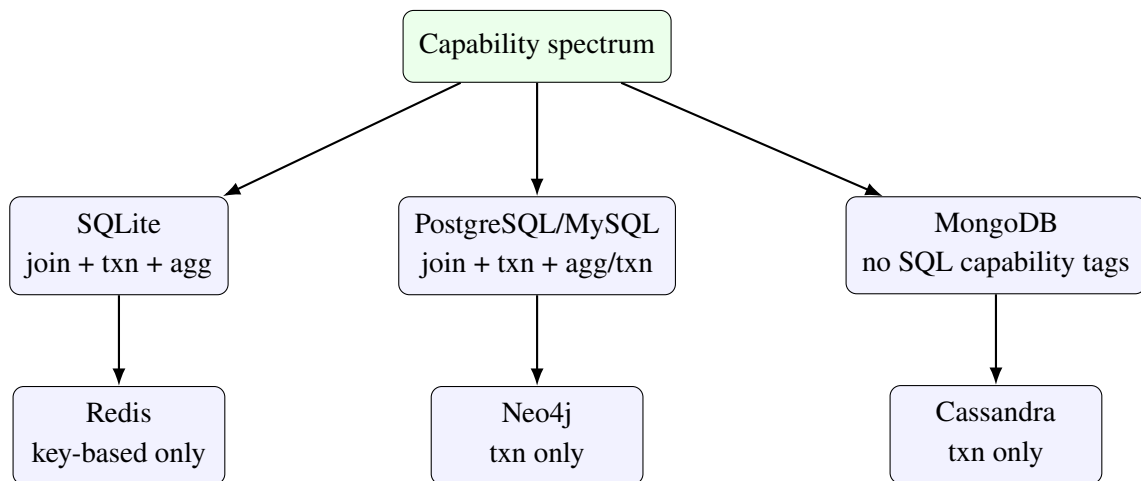
central architectural claim.

## 6.7. Wide-column scenario: Cassandra

This scenario is grounded in wide-column theory. Although Cassandra offers a CQL syntax reminiscent of SQL, the actual access model is governed by partition keys and the distributed organisation of data. The library therefore cannot treat Cassandra as an ordinary SQL backend. The mere existence of a `WHERE` clause is not enough; it must also remain compatible with partition logic.

CassandraDB and CassandraLiveDB declare `supports_transactions`, but not `supports_joins`. This is highly revealing. On the one hand, the backend remains part of the same ORM library. On the other hand, the contract categorically refuses operations that would be misleading or expensive in a wide-column environment. The unit and live tests in `tests/unit/test_cassandra_connector.cpp` and `tests/integration/test_cassandra_live.cpp` provide concrete verification of this stricter operational profile.

For that reason, Cassandra is the best example that the extensibility of the library is not based on false equivalence, but on formally described limits of admissible operations.



**Figure 12. Comparative capability profile of the backends in the repository**

Figure 12 synthesises one of the most important observations of the chapter: the library does not impose the same capability profile on every backend. On the contrary, the differences are used as first-class architectural information.



## 6.8. Comparison between test evidence and backend contract

**Table 10. Backends, capability profile, and available test evidence**

| Backend    | Capability profile                        | Unit/contract test                                                 | Integration/live test                          |
|------------|-------------------------------------------|--------------------------------------------------------------------|------------------------------------------------|
| SQLite     | joins,<br>transactions,<br>aggregation    | capability asserts<br>in tests/integr<br>ation/test_sql<br>ite.cpp | tests/integration/test_sqlite.cpp              |
| PostgreSQL | joins,<br>transactions,<br>aggregation    | tests/unit/tes<br>t_postgresql_c<br>onnector.cpp                   | tests/integration/test_postgresql_<br>live.cpp |
| MySQL      | joins,<br>transactions,<br>aggregation    | tests/unit/tes<br>t_mysql_connec<br>tor.cpp                        | tests/integration/test_mysql_live.<br>cpp      |
| MongoDB    | no SQL<br>capability<br>tags              | tests/unit/tes<br>t_mongodb_conn<br>ector.cpp                      | tests/integration/test_mongodb_liv<br>e.cpp    |
| Redis      | no joins/<br>transactions/<br>aggregation | tests/unit/tes<br>t_redis_connec<br>tor.cpp                        | tests/integration/test_redis_live.<br>cpp      |
| Neo4j      | transactions                              | tests/unit/tes<br>t_neo4j_connec<br>tor.cpp                        | tests/integration/test_neo4j_live.<br>cpp      |
| Cassandra  | transactions                              | tests/unit/tes<br>t_cassandra_co<br>nnector.cpp                    | tests/integration/test_cassandra_l<br>ive.cpp  |

Table 10 is important because it translates the comparison from the level of narrative analysis to the level of verifiable repository evidence. For every backend there is at least one explicit contract-level or live-level correlate. This reduces the risk that the chapter becomes a declarative survey without technical support.

## 6.9. Comparative synthesis

The comparison across backends shows that the library succeeds in preserving a shared logical model at the level of entity types, fields, placeholders, and query IR. Differences appear at the level of capabilities and physical data organisation. This is the central conclusion of the chapter: a successful multi-backend ORM architecture does not erase differences between storage models, but makes them controllable through a formal connector contract.

**Table 11. Comparison of backend materialisation**

| Backend          | Primary structure | Logical interpretation           | Representative limitation                                     |
|------------------|-------------------|----------------------------------|---------------------------------------------------------------|
| SQLite           | table             | full SQL ORM interpretation      | standard relational discipline and rich query expressiveness  |
|                  | table             | portable SQL IR                  | different binding models and dialect details                  |
| PostgreSQL/MySQL |                   |                                  |                                                               |
| MongoDB          | document          | filter/projection interpretation | only partial equivalence with SQL operations                  |
| Redis            | key/value, hash   | key-based entity access          | lookup mostly by primary key and no join semantics            |
| Neo4j            | node/properties   | graph-pattern interpretation     | graph-specific query semantics rather than SQL join behaviour |
| Cassandra        | wide-column       | CQL with key                     | partition-driven access and restricted                        |
|                  | row               | constraints                      | predicate profile                                             |

As a result, the multi-backend design of the library can be defended with a more precise formulation. The system does not promise a “universal database through one API”, but rather a *unified logical layer with explicit backend boundaries*. This formulation is both engineering-honest and scientifically grounded.

## VII. VERIFICATION AND EXTENSIBILITY THROUGH NEW CONNECTORS

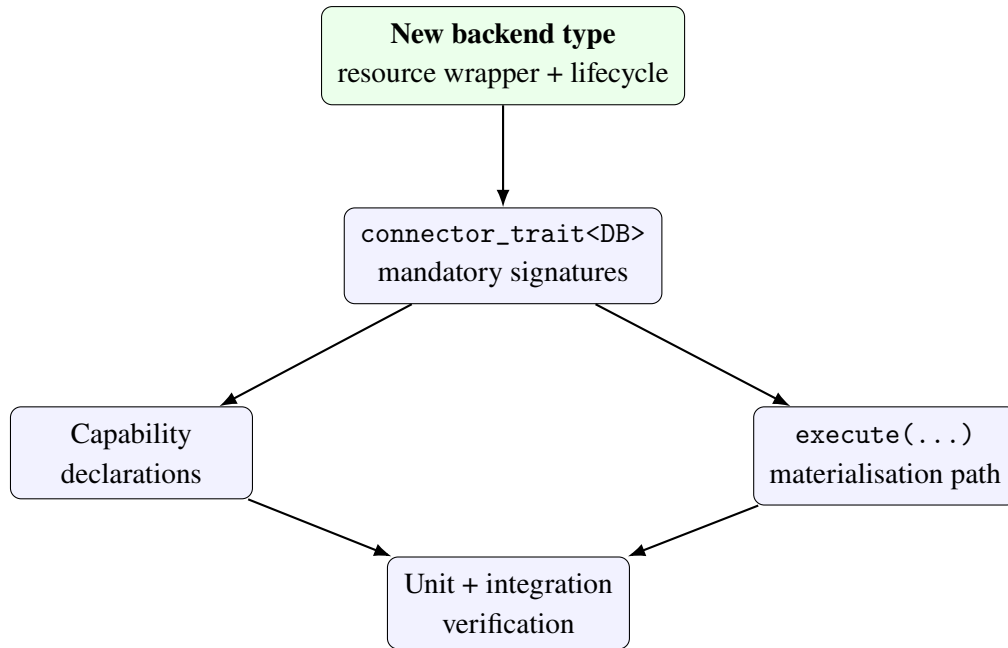
One of the central claims of the project is that the library is both verifiable and extensible. This requires not merely the presence of tests, but a clearly defined connector contract that allows a new backend to be added without changing the query API and without breaking compile-time guarantees.

The previous chapters showed that the library already works across relational and non-relational backends. The present chapter formalises how this is possible and how the next backend should be introduced. In other words, extensibility ceases here to be a general quality of the design and becomes a methodology with concrete requirements, signatures, testing criteria, and levels of completeness.

### 7.1. Formal contract of the connector layer

A connector is described through a DB type and a `connector_trait<DB>` specialisation. This is a structural rather than inheritance-based contract. There is no common virtual interface such as `Connector`. The reason is principled: different backends expose different capabilities and limitations, and those differences must remain visible at compile time. A virtual hierarchy would erase too many distinctions and would move part of validation toward runtime.

A minimally working connector must provide `wire_type`, `cursor_type`, and suitable `execute(...)` entry points. Additional abilities are denoted through capability tags. In this way the contract remains both strict and extensible. There is no “magic” registration layer in this construction: the compiler sees the specialisation directly and can verify whether it satisfies the `is_connector` concept.



**Figure 13. Architectural workflow for adding a new connector**

Figure 13 shows that adding a backend is not a one-step action. It begins with a clearly defined resource wrapper, continues with the trait contract, the capability profile, and the execution materialisation, and ends only after test verification. This is essential for the thesis because it places extensibility into a disciplined engineering process.

## 7.2. Mandatory elements of a minimal connector

In practical terms, a backend is considered minimally integrated if:

1. a type exists that represents the backend connection wrapper;
2. a `connector_trait<DB>` specialisation exists for that type;
3. the specialisation defines `template<typenameT>structwire_type;`
4. the specialisation defines `cursor_type;`
5. the specialisation can execute at least the basic ORM query forms it claims to support.

This means that a “connector” is not merely a rendering function, but a full description of how C++ types and the query IR pass into a concrete backend. If even one of these elements is missing, the added backend is not architecturally complete, regardless of whether some subset of queries can be made to run.

**Table 12. Mandatory and optional connector elements**

| Element               | Role                                                                 |
|-----------------------|----------------------------------------------------------------------|
| DB type               | wrapper over a backend handle, session, or mock connection           |
| connector_trait<DB>   | central contract specialisation                                      |
| >                     |                                                                      |
| template<typenameT>   | maps a C++ type into its wire representation                         |
| >structwire_type      |                                                                      |
| cursor_type           | describes result traversal or cursor lifecycle                       |
| execute(...)          | materialise the query IR into backend behaviour                      |
| overloads             |                                                                      |
| Capability tags       | restrict admissible operations and lifecycle policies                |
| begin/commit/rollback | required when transaction support is claimed                         |
| ck                    |                                                                      |
| ddl_for(...)          | required for schema-aware integration and migrate<DB>                |
| render_constexpr(...) | optional compile-time materialisation hook for specialised scenarios |
| .)                    |                                                                      |

### 7.3. Typing, signatures, and naming

The connector contract is strict also in terms of typing. `wire_type<T>::type` must be a stable compile-time function of `T`. `cursor_type` must model realistic result iteration or at least clearly indicate mock behaviour. `execute(...)` overloads must accept `DB&` as the first argument and return `orm::result<...>` for SELECT forms or `result<std::tuple<>>` for write operations.

Naming of the connectors in the repository follows a consistent convention. “Local” or unit-friendly connectors use names such as `SQLiteDB`, `PostgreSQLDB`, `MySQLDB`, `MongoDB`, `RedisDB`, `Neo4jDB`, and `CassandraDB`. Real live variants are distinguished explicitly through the suffix `LiveDB`. This is not a minor stylistic choice, but part of contract clarity: from the type name alone it is clear whether a given connector is a mockable adapter, an in-memory/local wrapper, or a real driver-backed backend.

Absence of inheritance must also be regarded as a formal rule rather than an implementation detail. A new connector should not derive from a common “ORM base class”, because that would mix backend-specific logic with runtime polymorphism. The correct

extension mechanism is specialisation of `connector_trait<DB>`.

## 7.4. Capability mechanism and compile-time restrictions

Capability tags are the primary way in which the library distinguishes backends by admissible operations. If `connector_trait<DB>` does not declare `supports_joins`, then an attempt to execute a join-based query is architecturally invalid and should be rejected already at compilation. The same reasoning applies to transactions, aggregation, and concurrent execution.

From the standpoint of verification, this is important because correctness no longer depends solely on tests. Part of the contract is directly checkable by the compiler. The tests then no longer prove that the contract exists, but that concrete specialisations apply it consistently. This is exactly why `tests/unit/test_mongodb_connector.cpp` and `tests/unit/test_redis_connector.cpp` validate the absence of `supports_joins`, `supports_transactions`, and `supports_aggregation` for those connectors.

This negative verification is just as important as positive verification. Proving that a backend *does not* support a given operation is as important as proving that it does support another.

## 7.5. Minimal connector skeleton and operational connector skeleton

In the repository, `MockDB` and `MockMigrateDB` are especially valuable because they act as connector archetypes. The former shows the operational contract for query execution, while the latter shows how the same contract can be extended toward schema-aware migration scenarios.

```
1  template <>
2  struct connector_trait<MockDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions   = void;
6      using supports_aggregation    = void;
7      using supports_upsert         = void;
8      using supports_bulk_insert    = void;
9      using supports_concurrent_execute = void;
10
11     template <typename T>
12     struct wire_type { using type = T; };
13
14     struct cursor_type
15     {
```

```

16         [[nodiscard]] bool has_next() const noexcept { return false; }
17     };
18
19     template <typename Response, typename Joins, typename Wheres,
20             typename Limits, typename Groups, typename Orders>
21     static auto execute(MockDB& db,
22         select_query<Response, Joins, Wheres, Limits, Groups, Orders> q)
23         -> result<projected_type<Response>, Response>;
24 };

```

**Listing 9:** Excerpt from connector\_trait<MockDB>

Listing 9 is revealing because it gathers in compact form almost all mandatory elements: capability profile, wire\_type, cursor\_type, and query-specific execute(...) signatures. Moreover, the connector uses separate overloads for select\_query, insert\_query, update\_query, and delete\_query, making the mapping toward backend behaviour explicit.

```

1  template <>
2  struct connector_trait<MockMigrateDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation     = void;
7      using supports_concurrent_execute = void;
8
9      template <typename T>
10     struct wire_type { using type = T; };
11
12     template <typename... Args>
13     static auto execute(MockMigrateDB& db, Args&&... args)
14     {
15         return connector_trait<MockDB>::execute(
16             static_cast<MockDB&>(db), std::forward<Args>(args)...);
17     }
18
19     [[nodiscard]] static std::string ddl_for(const create_table_op& op);
20     [[nodiscard]] static std::string ddl_for(const add_column_op& op);
21 };

```

**Listing 10:** Excerpt from the schema-aware extension MockMigrateDB

Listing 10 shows how the operational connector skeleton can be extended. Query-execution logic is delegated to MockDB, but DDL hooks and transaction hooks are added. This is a good example of a progressive connector-maturity model.

## 7.6. Transactions, thread safety, and prepared queries

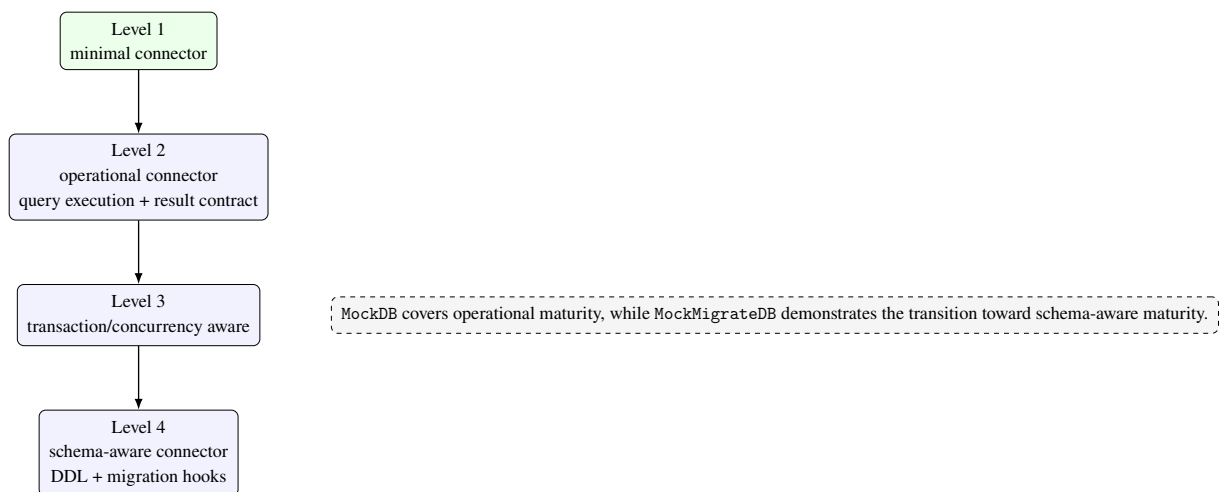
A mature backend contract is not exhausted by a single `SELECT`. If the connector declares `supports_transactions`, it is expected to support `begin`, `commit`, and `rollback` so that `transaction_guard<DB>` remains valid. If it declares `supports_concurrent_execute`, it must participate safely in `connection_pool<DB, N>`.

The prepared-query layer also imposes requirements regarding the stability of `DB` lifetime and the consistency of binding behaviour under repeated execution. Consequently, a mature connector should be evaluated not only by whether it renders correct syntax, but also by the operational contract around resources, transactions, and reuse. This is particularly important for backends whose driver lifecycle is more complex than that of `MockDB`.

## 7.7. Migrations and schema-aware integration

A connector that participates in `migrate<DB>` must also offer DDL extension points. In the repository this is demonstrated through `MockMigrateDB`. The example is especially useful because it shows how schema-aware behaviour can be layered on top of an already existing execution backend. In this way, the library formalises different levels of connector completeness.

It is important to note that schema-aware support is not mandatory for every backend. Some connectors may be entirely adequate for query-execution analysis without supporting automated migrations. But if a connector claims a fuller ORM integration, the presence of `ddl_for(...)` hooks and migration tests becomes mandatory.



**Figure 14. Maturity ladder for a new connector**

Figure 19 proposes a practical evaluation model. It is useful also for future development



of the library because it allows new backends to be positioned not simply as “connector present/absent”, but according to the level of their integration.

## 7.8. Methodology for adding a new backend

Adding a new backend follows a clear sequence. First, the backend type and its resource wrapper are defined. Next, `connector_trait<DB>` is implemented with the mandatory elements. Then capability declarations and the rules for rendering or executing the query IR are added. Finally, unit tests for the contract are written and, when possible, live integration tests are added for the real driver.

The architecturally correct order is important: one should not begin from runtime workarounds, but from formalising the compile-time boundaries and the mapping logic. If this order is violated, the result is usually a connector that “works” for one happy path but is not actually compatible with the general architecture.

**Table 13. Recommended verification matrix for a new connector**

| Verification layer             | What must be proven                                 | Representative repository correlate                                                                  |
|--------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Concept compliance             | <code>is_connector&lt;DB&gt;</code> is satisfied    | <code>tests/unit/test_mongodb_connector.cpp</code>                                                   |
| Capability honesty             | declared and undeclared capability tags are correct | <code>tests/unit/test_redis_connector.cpp</code> ,<br><code>tests/integration/test_sqlite.cpp</code> |
| Execution correctness          | the query IR is materialised correctly              | <code>tests/integration/test_mockdb.cpp</code> and the live backend tests                            |
| Resource/lifecycle correctness | transaction and concurrency hooks behave correctly  | <code>tests/unit/test_thread_safety.cpp</code>                                                       |
| Schema-aware correctness       | DDL hooks and migration diff behave correctly       | <code>tests/unit/test_schema_migration.cpp</code>                                                    |

## 7.9. Criteria for a “fully implemented and functioning” connector

On the basis of the code and tests in the repository, the following criteria can be formulated:

1. the connector satisfies the `is_connector` contract;
2. capability declarations correspond to real behaviour;
3. naming and typing conventions remain compatible with the other connectors;
4. resource lifecycle is correct and testable;
5. supported ORM operations are covered by unit tests;
6. when a live driver exists, integration tests run against a real database;
7. when schema-aware support is claimed, DDL hooks and migration tests are present.

These criteria are stricter than merely saying “there is a working adapter”, because they require consistency between architectural intent, API contract, and verified behaviour. That is precisely what distinguishes an ad hoc integration from a connector that truly belongs to the library.

## 7.10. Verification through tests

The test layer in the repository follows the structure of the architecture. Unit tests for the individual connectors validate `is_connector` compliance, capability declarations, and basic rendering. `tests/unit/test_thread_safety.cpp` checks `connection_pool`, `thread_local_db`, and `transaction_guard`. `tests/unit/test_schema_migration.cpp` validates the migration diff logic. `tests/integration/test_mockdb.cpp` and the live integration tests for individual backends prove that the abstraction also works during real execution.

Verification is therefore not concentrated in one place. It is distributed so that every architectural claim has at least one direct testing correlate. This turns connector extensibility from a declarative objective into a verifiable engineering practice.

## 7.11. Assessment of the existing connectors

SQLiteDB may be regarded as a reference mature implementation for a relational backend. PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB, and CassandraDB already demonstrate important parts of the contract and have varying degrees of live coverage. MockDB and MockMigrateDB play a special role as verification connectors: they are not merely test doubles, but instruments for formalising the rules of the architecture.

A broader conclusion follows from this: the extensibility of the library was not added afterwards, but is embedded in the very way the connector layer is defined. That is exactly what allows the project to continue toward new backends without breaking the formal character of the compile-time ORM architecture.

## VIII. LIMITATIONS AND FUTURE WORK

The current version of the library already demonstrates a working architecture for Compile-time Object-Relational Mapping across relational and non-relational backends, but scientific accuracy requires a clear distinction between what has already been achieved and what remains for future work. The repository contains a real query-IR layer, a capability-based connector contract, a migration prototype, thread-safety helpers, and a substantial set of unit and integration tests. At the same time, some subsystems still need to be extended or validated more broadly under production-like conditions.

Future development is not an arbitrary list of ideas. It follows directly from the architectural decisions analysed in the previous chapters. If the C++ model is the primary carrier of the logical schema, then fuller schema-aware tooling is a natural next step. If the connector layer is the central point of extensibility, then future work must refine the taxonomy of capabilities. If the query IR is truly the shared layer across distinct backends, then it must be exercised under richer operational scenarios and stricter performance constraints.

### 8.1. Strategic directions of development

From the perspective of the present thesis, future work can be grouped into four strategic directions. The first is *greater operational maturity* of the already existing backends. The second is *more complete automation of schema and metadata handling*. The third is *reduction of runtime overhead* through lower-level execution and tighter control of the wire-protocol layer. The fourth is *reduction of boilerplate* through deeper integration with newer C++ language facilities.

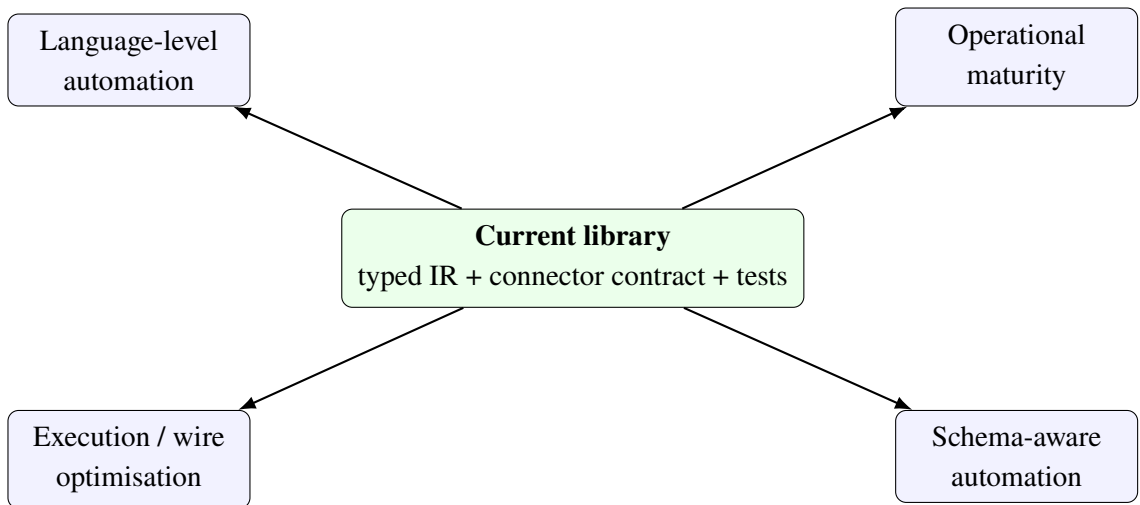


Figure 15. Four strategic directions for future development

Figure 15 is useful because it shows that future work is not one-dimensional. Some tasks aim at operational maturity, others at stronger compile-time automation, and still others at performance and runtime engineering. This distinction matters because the different directions require different success criteria.

## **8.2. Broader live backend coverage**

Although the repository already contains live integration tests for several backends, the maturity of the individual implementations is not identical. SQLite remains the most natural reference relational path because it combines query rendering, binding logic, and result hydration most tightly. For PostgreSQL, MySQL, MongoDB, Redis, Neo4j, and Cassandra, the next step is broader coverage of operational scenarios: richer type families, diagnostic paths under failure, connection lifecycle policies, and behaviour under more demanding query patterns.

This direction matters not only as an engineering improvement, but also as a scientific validation of the limits of the shared query IR. Only broader empirical coverage can show how far one and the same logical layer remains adequate across distinct storage systems. A join-oriented logic that is natural for PostgreSQL, for instance, carries a very different operational meaning in Neo4j or Cassandra. Live coverage is therefore not merely a testing discipline, but an instrument for studying architectural applicability.

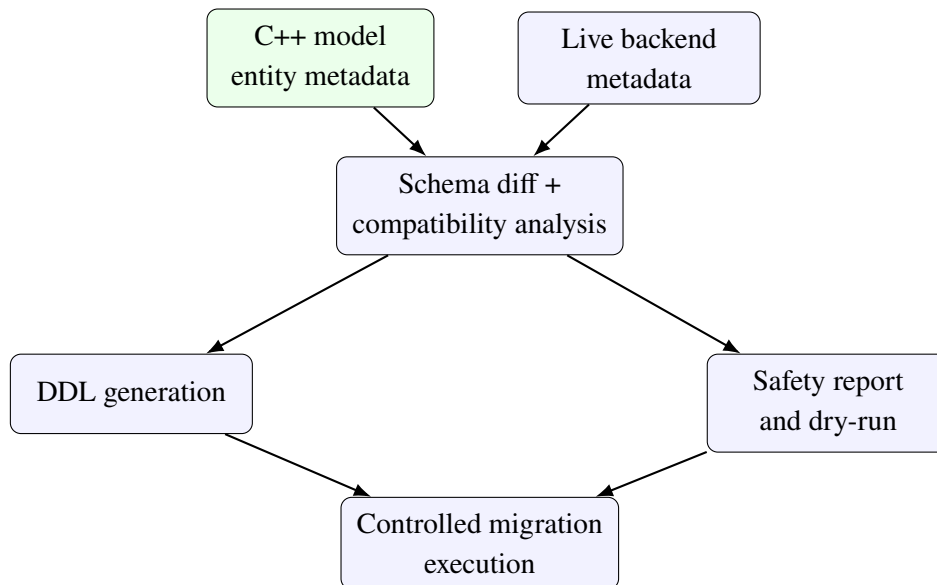
In addition, live backend coverage should include negative paths: connection-establishment failures, partially invalid parameter sets, driver-specific timeout scenarios, and behaviour under interrupted connections. These questions are often underestimated in early ORM systems, but in a multi-backend context they are essential for real adoption.

## **8.3. Full schema-aware integration and runtime introspection**

The prototype migration layer is sufficient to show how the C++ entity model can participate in schema evolution. The next natural direction is live schema introspection, metadata extraction from drivers, and safe execution of DDL operations. This is especially important for relational backends, but it also matters for document and wide-column systems, where the notion of schema may be partial or more flexible.

Further work in this direction must clarify the boundary between compile-time

guarantees and runtime facts about a deployed system. The compiler can validate query structure, but it cannot validate the real state of a production schema without supplementary runtime information. A future migration layer should therefore combine two sources of truth: the entity description in C++ and the metadata picture extracted from the live database.



**Figure 16. Target direction for schema-aware tooling beyond the current migration prototype**

Figure 16 shows that the future migration pipeline must be bidirectional: it must read both from the compile-time description and from the live system. This is the natural next step if the library is to be used not only experimentally, but also as a practical development tool.

## 8.4. Thread safety and connection lifecycle

The existing helper layer for connection pooling, thread-local access, and transaction guards already provides an architectural basis for multi-threaded use. Future work must answer more precise questions: how exactly `supports_concurrent_execute` should be defined for different drivers; when a connection is safe to share; and which policies should govern reconnection, shutdown, and deferred release of resources.

There is also an important research dimension here: different backends have different concurrency semantics, and therefore compile-time capability annotation must be precise enough to avoid misleading promises. It is entirely possible that future versions of the library will split the current `supports_concurrent_execute` into finer-grained capabilities such as safe parallel reads, parallel execution with isolated connection handles, or strict serialisation policies.

In practical terms, this means that the thread-safety helper layer should not remain an isolated convenience module. On the contrary, it should become an architectural continuation of the connector contract and participate in the full assessment of backend maturity.

## **8.5. Low-level execution and wire-protocol optimisation**

The experimental layer in `WireProtocol/wire_protocol.hpp` points toward future directions such as `io_uring` / IOCP awaitables, `zero_copy_result`, `batch_insert`, and `make_constexpr_sql`. These components are not yet a production-ready communication stack, but they outline a possible path by which the library may move beyond the classical synchronous driver model.

Especially promising are zero-copy result handling and compile-time SQL generation for connectors that can support it. Both would reduce runtime overhead further without sacrificing the typed architecture. This matters because a common criticism of heavily typed abstraction layers is that they hide excessive runtime cost. Future work should demonstrate that compile-time expressiveness and low overhead are not necessarily opposing goals.

Moreover, performance should not be viewed only through latency or throughput. It also includes memory layout of results, the number of temporary allocations, repeated reuse of prepared-query artefacts, and the quality of driver-level batching strategies.

## **8.6. Integration with C++26 reflection and further automation**

Support for C++26 reflection has already been conceptually prepared through build-time detection and helper paths, and `tests/unit/test_cpp26_reflection.cpp` shows the presence of an early verification route. The public API, however, still relies on explicit string literal names in `property<T,Name>`. Future work should turn the reflection path into a stable public mechanism that can infer field names automatically while preserving backward compatibility.

Such a development would reduce boilerplate and make even more explicit the idea that the C++ model is the primary source of the logical schema. From a research perspective, it would also move the library closer to a model-first style in which much of the metadata is derived automatically rather than duplicated manually.

There is, however, an important boundary here. Reflection should not weaken the clarity of the compile-time contract. If automation reduces type transparency or makes compiler

diagnostics harder to understand, it may undermine one of the principal values of the library. This direction must therefore be pursued carefully.

## 8.7. Extension of the connector contract and capability taxonomy

As new backends are added, it may become necessary to distinguish more finely between classes of capability. Future versions of the library may need to separate graph traversal support, partial document embedding support, schema introspection support, streaming result support, or explicit batching support. This direction follows naturally from the current trait-based architecture.

An important rule must be preserved, however: new capability tags should never be decorative. They must carry real compile-time meaning and constrain inadmissible operations. If a capability tag has no architectural consequence, it adds noise instead of value and makes the contract less precise.

**Table 14. Priority directions for future development**

| Direction                   | Affected subsystems                               | Expected effect                                            | Principal challenge                             |
|-----------------------------|---------------------------------------------------|------------------------------------------------------------|-------------------------------------------------|
| Live backend maturity       | connectors, integration tests, error paths        | higher practical reliability                               | different operational semantics across backends |
| Schema-aware automation     | migrate<DB>, DDL hooks, metadata introspection    | a more production-oriented ORM integration                 | combining compile-time and runtime truth        |
| Execution optimisation      | wire-protocol layer, prepared queries, batching   | lower runtime overhead                                     | preserving the typed abstraction                |
| Reflection-driven modelling | entity layer, metadata extraction, API ergonomics | less boilerplate and a more automatic model-first workflow | balancing automation against transparency       |
| Capability refinement       | connector contract and static checks              | a more precise contract for future backends                | avoiding decorative capability tags             |



## **8.8. Need for stricter empirical evaluation**

A future version of the work should also include a more systematic benchmarking layer. It should compare not only separate backends, but also different execution modes inside the library itself — direct execution, `prepare()` + reuse, batch-insert scenarios, and distinct forms of result materialisation. Such benchmarking would be valuable both engineering-wise and scientifically because it would reveal the concrete cost of the different architectural choices.

Alongside this, it would be useful to add a formalised evaluation of error diagnostics. One of the strengths of a compile-time ORM design lies precisely in early fault localisation. The quality of error messages and the clarity of compile-time failure paths are therefore themselves valid objects of investigation.

## **8.9. Summary**

The future of the library does not begin from an empty slate. On the contrary, it builds on an architecture that is already general enough to absorb new backends and strict enough to enforce formal limits. This is what makes the work suitable both for continued engineering development and for further academic investigation into compile-time modelling of data access.

The most important conclusion of the present chapter is that future work should not be understood as a repair of an incoherent prototype. It is the natural unfolding of an already defensible architectural foundation. That difference matters: the library has reached a stage at which extension is a matter of disciplined development rather than conceptual reinvention.

## IX. CONCLUSION

This thesis presented the design, implementation, and analysis of a Compile-time Object-Relational Mapping library for C++, intended for both relational and non-relational databases. At the centre of the work stands the idea that query structure, the logical data model, and a significant portion of backend admissibility constraints can be described and validated through the type system of the language.

In doing so, the thesis addresses a concrete problem of contemporary practice: the mismatch between strongly typed application code and the heterogeneous storage models that such code must use. Classical ORM solutions are strongest within the relational world, while purely driver-oriented approaches often move too much correctness toward runtime. The present work showed that between these two poles there exists a viable architectural position — a compile-time ORM layer that is practical, formally organised, and sufficiently honest about backend differences.

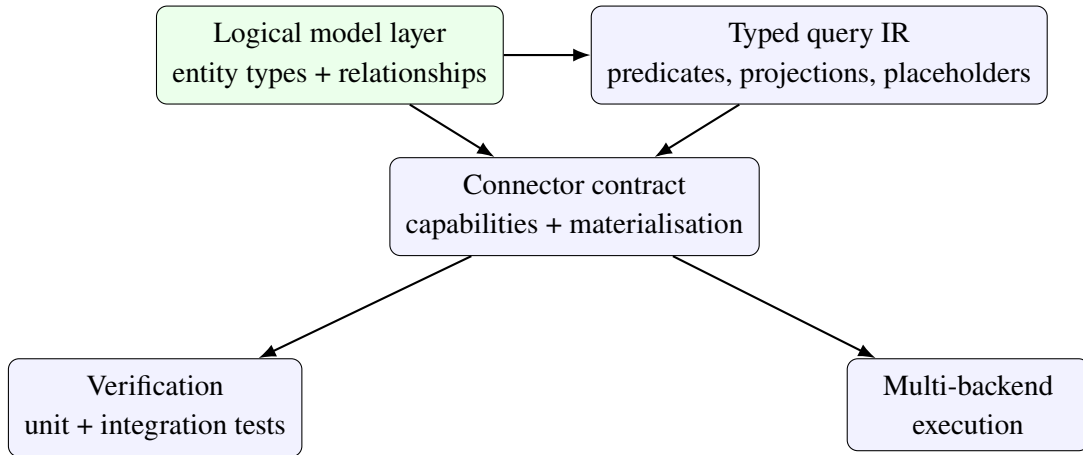
### 9.1. Summary of the solved problem

The task was not merely to build another convenient query builder. The goal was to design a library in which:

1. the logical data model is described through C++ types;
2. queries are represented as a typed IR rather than as strings;
3. backends are integrated through a formal connector contract;
4. differences between storage models are managed through capability and constraint mechanisms;
5. correctness is supported simultaneously by compile-time checks and by a testing infrastructure.

Within the thesis, each of these sub-goals was addressed through concrete architectural and implementation decisions. The entity layer introduced typed properties and relationships, the `store_as` policy, and traits for logical container naming. The query builder was organised around a typed intermediate representation. The connector layer was formalised through con

nector\_trait<DB>, and the verification strategy was supported by unit and integration tests across multiple backends.



**Figure 17. Synthesis of the principal construction of the developed library**

Figure 17 summarises the overall logic of the thesis. It shows that the library is not a collection of isolated techniques, but a coherent system in which the model layer, query IR, connector contract, verification, and multi-backend execution support one another.

## 9.2. Summary of achieved results

The work showed that C++ provides sufficiently strong mechanisms — templates, `constexpr`, concepts, and trait specialisations — for building an ORM layer in which queries are not described as strings. Instead, they are represented through a typed query IR that can be analysed at compile time and materialised differently depending on the chosen backend.

At the level of data modelling, the library demonstrates how `property`, `relationship`, `store_as`, and `table_name_trait` can serve as a unified logical layer. That layer is not restricted to relational interpretation, but acts as a shared description of data and relationships. It is precisely this that allows one and the same application-level structure to be carried toward tables, documents, key-value structures, graph nodes, or wide-column rows.

At the level of architecture, the connector trait is formalised as the central extensibility mechanism. Through it, the library adapts one and the same query IR to SQL, BSON-like filters, Redis commands, Cypher, and CQL. The capability mechanism complements that architecture by making inadmissible operations visible already at compilation.

At the level of verification, the work relies on a systematic set of unit and integration tests covering the entity layer, the query builder, prepared queries, migrations, thread safety, and

concrete connectors. Architectural claims are therefore supported by real evidence from code and testing infrastructure.

### **9.3. Scientific and practical contributions**

The first contribution of the work is the demonstration that a practically useful Compile-time Object-Relational Mapping library can be realised in standard C++. This is achieved without external code generation, without a virtual connector interface, and without forcing the user outside idiomatic C++.

The second contribution lies in clarifying the relationship between the C++ entity description and the physical organisation of stored data. The thesis shows that the C++ model can be treated as the primary carrier of the logical schema, while the concrete backend determines the mode of physical materialisation. This claim has value not only for ORM libraries, but more broadly for systems that try to combine strong static typing with heterogeneous data sources.

The third contribution is the formalisation of a connector contract that combines extensibility with strict compile-time discipline. This contract makes it possible to extend the library with new backends without breaking the guarantees already embedded in the query API. It is exactly here that the work differs from approaches that provide a shared interface but conceal critical backend differences.

The fourth contribution is the demonstration that verification of such a library must be multi-layered. It is not sufficient merely to “have tests” or merely to “have compile-time checks”. What is needed is a combination of the two: concept-level constraints, capability assertions, query-rendering tests, live integration scenarios, and migration/thread-safety validation.

**Table 15. Relationship between the thesis goals and the realised results**

| <b>Goal</b>                    | <b>Realised mechanism</b>                | <b>Evidence basis</b>                                                       |
|--------------------------------|------------------------------------------|-----------------------------------------------------------------------------|
| Typed data model               | property, relationship, table_name_trait | entity headers and the property/relationship unit tests                     |
| Typed query layer              | query builders and typed IR              | query headers, tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp |
| Extensible multi-backend layer | connector contract and capability tags   | connector headers and backend-specific unit/live tests                      |
| Schema-aware direction         | migrate<DB> and DDL hooks                | tests/unit/test_schema_migration.cpp                                        |
| Safe operational use           | thread-safety and transaction helpers    | tests/unit/test_thread_safety.cpp                                           |

Table 15 shows that the thesis does not remain at the level of a declarative design document. For each principal goal there is a concrete realised mechanism and at least one direct technical correlate in the repository.

#### **9.4. Principal limitations and the boundaries of generalisation**

The work also shows clearly that a multi-backend ORM abstraction has limits. Not every storage model offers the same operations, and therefore the shared query IR cannot be interpreted identically everywhere. That is precisely why capability and constraint mechanisms are so important. They do not weaken the library; they make it correct with respect to different backend families.

In addition, although the architectural foundation for migrations, thread safety, and lower-level execution support is already present, these subsystems remain natural areas for future development and broader empirical expansion. This means that the work should be evaluated as a completed architectural and implementation demonstration, but not as the last word on compile-time data access.

From an academic point of view, it is also important to stress another boundary. The present work does not prove that compile-time ORM is universally the best approach for all

applications. It proves something more precise: that for a class of systems requiring clear static structure, high type safety, and controlled multi-backend extensibility, this approach is viable and engineering-sound.

## **9.5. Significance for practice and for future research**

The practical value of the work lies in the fact that the library already provides a stable foundation for further engineering development. The entity description, query IR, connector contract, and verification strategy are sufficiently clear to allow disciplined addition of new backends and operational capabilities.

Its research value is related to the broader question of the role of the type system in modelling data access. The thesis shows that the C++ type system can serve not only for describing data and algorithms, but also for formalising architectural boundaries, admissibility of operations, and the transition between logical and physical layers. This places the library in a wider context that extends beyond ORM in the narrow sense.

## **9.6. Closing remarks**

The analysis confirms that the C++ type system is sufficiently powerful to serve not only for modelling data and algorithms, but also for formalising access to persistent systems. The combination of a static entity model, query IR, a trait-based connector layer, and systematic verification demonstrates a viable path toward libraries that are expressive, safe, and extensible at the same time.

In this way, the thesis achieves both an engineering and a research goal: it delivers a working library while also formulating principles that can guide future development of compile-time data-access layers. The most important conclusion is that the compile-time ORM approach is not merely a theoretical exercise, but a real engineering alternative for systems that require rigour, clarity, and extensibility across multiple storage models.

## REFERENCES

- [1] Code Synthesis Tools CC. ODB: C++ object persistence framework. <https://www.codesynthesis.com/products/odb/>, 2010. Online documentation.
- [2] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] D. Richard Hipp. SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine. <https://www.sqlite.org/>, 2000. Online documentation.
- [4] D. Richard Hipp. SQLite C/C++ interface – an introduction. <https://www.sqlite.org/cintro.html>, 2000. Online documentation.
- [5] Maciej Hryniuk and Mateusz Bao. SOCI: The c++ database access library. <https://soci.sourceforge.net/>, 2004. Online documentation.
- [6] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2024 – programming language C++. International standard, International Organization for Standardization, 2024.
- [7] Oracle Corporation. MySQL connector/C – C client library for MySQL. <https://dev.mysql.com/doc/c-api/en/>, 2000. Online documentation.
- [8] The PostgreSQL Global Development Group. libpq — C library for PostgreSQL. <https://www.postgresql.org/docs/current/libpq.html>, 1996. Online documentation.
- [9] Roland Witt. sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++. <https://github.com/rbock/sqlpp11>, 2014. GitHub repository.

## **X. GLOSSARY OF ABBREVIATIONS AND TERMS**

The present section serves two functions. On the one hand, it summarises the abbreviations and terms used throughout the thesis in order to make the text easier to follow consistently. On the other hand, it fixes the precise meaning in which several concepts are used within the work. This matters because words such as “model”, “connector”, “migration”, or “capability” can carry different meanings depending on context.

### **10.1. Basic abbreviations**

Table 16 summarises the principal abbreviations and terms used in the thesis.



**Table 16. Glossary of abbreviations**

| <b>Abbreviation</b> | <b>Full form</b>                       | <b>Context</b>                                         |
|---------------------|----------------------------------------|--------------------------------------------------------|
| <b>API</b>          | Application Programming Interface      | Programming interface between software components      |
| <b>BSON</b>         | Binary JSON                            | Binary serialisation format used by MongoDB            |
| <b>CQL</b>          | Cassandra Query Language               | Query language for Apache Cassandra                    |
| <b>CRUD</b>         | Create, Read, Update, Delete           | The four fundamental data operations                   |
| <b>DDL</b>          | Data Definition Language               | Language for schema definition and evolution           |
| <b>IR</b>           | Intermediate Representation            | Intermediate representation of query structure         |
| <b>ORM</b>          | Object-Relational Mapping              | Mapping between an object model and persistent storage |
| <b>PFR</b>          | Plain Fields Reflection                | Boost . PFR mechanism for aggregate inspection         |
| <b>RAII</b>         | Resource Acquisition Is Initialization | C++ idiom for deterministic resource management        |
| <b>RDBMS</b>        | Relational Database Management System  | Engine for relational data storage                     |
| <b>SFINAE</b>       | Substitution Failure Is Not An Error   | Classical C++ template-constraining mechanism          |
| <b>SQL</b>          | Structured Query Language              | Standard language for relational queries               |
| <b>TMP</b>          | Template Metaprogramming               | Compile-time computation through templates             |
| <b>WG21</b>         | Working Group 21                       | ISO working group responsible for the C++ standard     |

## 10.2. Working definitions of key concepts

Within the thesis, several terms are used in a more specific sense than in everyday technical language. Table 17 fixes that sense so that architectural and general meanings are not inadvertently mixed.

**Table 17. Working definitions of the key concepts used in the thesis**

| Concept              | Working definition                                                                                                                    |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Logical model        | the compile-time description of entity types, their properties, and their relationships, independent of a concrete physical backend   |
| Query IR             | the typed intermediate representation of a query before materialisation into SQL, a BSON-like filter, a Redis command, Cypher, or CQL |
| Connector contract   | the formal set of requirements imposed on <code>connector_trait&lt;DB&gt;</code> through which a backend participates in the system   |
| Capability tag       | a compile-time marker that constrains admissible operations and expresses architectural abilities of the backend                      |
| Prepared query       | an object that binds a previously constructed query IR to a concrete <code>db&lt;DB&gt;</code> instance for repeated execution        |
| Schema-aware support | a level of integration at which the library can work with schema metadata, DDL hooks, and migration-diff logic                        |
| Hydration            | materialisation of backend results into a typed C++ result object                                                                     |
| Verification         | the joint role of compile-time constraints, unit tests, and integration tests in demonstrating correctness                            |

### 10.3. Notation and naming used in the text

The thesis also uses a consistent notation for types, traits, and backend roles. This notation is not accidental; it reflects the real structure of the repository and makes the connection between the text and the code easier to follow.

**Table 18. Notation and naming conventions used in the text**

| Notation                          | Meaning in the thesis                                                                                    |
|-----------------------------------|----------------------------------------------------------------------------------------------------------|
| DB                                | a generic backend type used as a parameter in traits, helpers, and the <code>db&lt;DB&gt;</code> wrapper |
| <code>db&lt;DB&gt;</code>         | the facade type that exposes the user-facing query-execution interface                                   |
| <code>connector_trait</code>      | the specialisation that describes how a concrete backend executes the query IR                           |
| <code>wire_type&lt;T&gt;</code>   | the mapping from a C++ type into a backend-specific wire representation                                  |
| <code>cursor_type</code>          | the abstraction describing result traversal or result-lifecycle management                               |
| <code>field&lt;...&gt;</code>     | a compile-time designation of a selected property in the query API                                       |
| <code>Placeholder&lt;T&gt;</code> | an anonymous runtime parameter in the query IR                                                           |
| <code>*LiveDB</code>              | the naming convention for connectors that use a real driver-backed backend rather than a mock variant    |

## 10.4. Repository context of the most frequently cited artefacts

Because the thesis is closely tied to the real repository, it is also useful to fix the role of the most frequently cited groups of files. Table 19 serves as a compact guide to where the main evidence sources reside in the project.

**Table 19. Map of the principal repository artefacts used in the text**

| Path or file group            | Role                                   | How it is used in the thesis                                                        |
|-------------------------------|----------------------------------------|-------------------------------------------------------------------------------------|
| lib/include/ORM/entity/       | entity model                           | used as the source for property, relationship, and the logical description of data  |
| lib/include/ORM/query/        | query IR and fluent API                | used for analysing select, insert, update, delete, and the query-composition layer  |
| lib/include/ORM/connector/    | connector contract and execution layer | used to formalise the capability mechanism, prepared queries, and the db<DB> facade |
| lib/include/ORM/db/migration/ | migration and schema-aware logic       | used as the basis for analysing DDL hooks and the migration-diff pipeline           |
| tests/unit/                   | unit verification                      | serves as the evidence layer for local correctness of the individual subsystems     |
| tests/integration/            | integration verification               | serves to demonstrate that the abstraction works under real or mock query execution |
| doc/v2/bg/ and doc/v2/en/     | documentation layer                    | used for the synchronous academic presentation of architecture and results          |

## 10.5. Conclusion

The extended glossary shows that terminological consistency is an inseparable part of the quality of the present thesis. When an architecture is strongly typed and oriented toward multiple backend families, precise use of terms is not merely a stylistic question, but a condition for correct understanding of the entire argument.

## XI. MAP OF VERIFICATION SCENARIOS AND REPOSITORY ARTEFACTS

The present appendix systematises the relationship between the architectural claims made in the thesis and the concrete artefacts in the repository. Its goal is not to duplicate arguments already given in the main text, but to provide a compact and verifiable map of where the principal evidence for correctness resides in the code and testing infrastructure.

Such an appendix is especially useful in the context of the present work because the library is not merely a conceptual prototype. It is implemented as a real CMake project with source code, unit tests, integration tests, and conditional live tests for different backend families. It is exactly this material structure that allows the thesis to move beyond the level of “architectural intention” and to be defended as an engineering system with verifiable evidence.

### 11.1. Overall structure of the evidence layer

Verification in the repository follows the architectural structure of the library. Production code is concentrated primarily under `lib/include/ORM/`, while tests are divided between a local unit layer and an integration/live layer. The CMake files register those tests differently depending on whether a backend is always available, conditionally available, or dependent on an external environment.

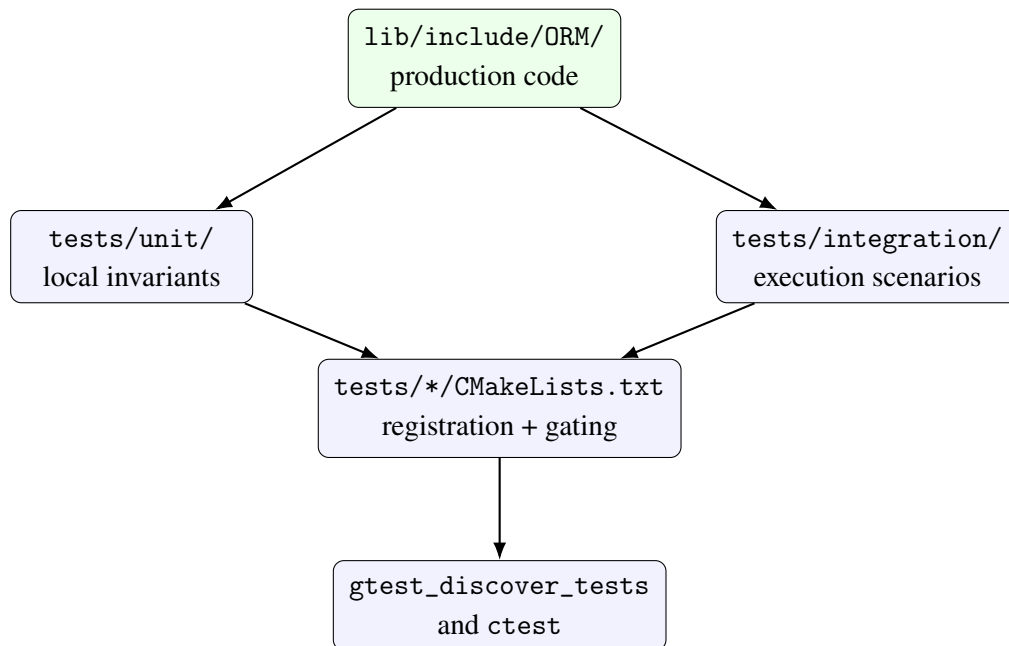


Figure 18. Verification layer of the repository and its flow toward executable tests

Figure 18 shows why verification should not be perceived as a single test package. It is composed of several levels: local properties of types, contract behaviour of connectors, integration scenarios for query execution, and a build-level mechanism that decides which tests are available in a given environment.

## 11.2. Catalogue of the principal unit tests

The unit tests cover two different categories of knowledge. The first is related to the logical model and the generic mechanisms of the library. The second is related to backend-specific connectors and their capability profiles.

**Table 20. Principal unit tests for the generic and core subsystems**

| File                      | Subsystem                    | What it validates                                                             |
|---------------------------|------------------------------|-------------------------------------------------------------------------------|
| test_types.cpp            | foundational type aliases    | correctness of public aliases, including chrono-related types                 |
| test_property.cpp         | entity property layer        | compile-time model of scalar fields and type-trait behaviour                  |
| test_relationships.cpp    | entity relationship layer    | inference logic for relationships and storage-strategy behaviour              |
| test_query.cpp            | query IR and fluent API      | correctness of select, insert, update, delete q, and derived query structures |
| test_thread_safety.cpp    | concurrency helpers          | behaviour of connection pooling, thread-local access, and transaction guards  |
| test_wire_protocol.cpp    | experimental execution layer | lower-level compile-time and wire-oriented mechanisms                         |
| test_cpp26_reflection.cpp | future language integration  | early path toward reflection-driven automation                                |
| test_schema_migration.cpp | migration layer              | DDL generation, diff logic, and state-machine behaviour                       |

Table 20 is important because it shows that the unit layer is not exhausted by connector tests. It also covers the generic architectural elements on which all backend integrations later depend.

**Table 21. Backend-oriented unit tests and their contract focus**

| File                                       | Backend focus | Type of evidence                                                                         |
|--------------------------------------------|---------------|------------------------------------------------------------------------------------------|
| <code>test_postgresql_connector.cpp</code> | PostgreSQLDB  | <code>is_connector</code> compliance, capability assertions, and SQL parameter rendering |
| <code>test_mysql_connector.cpp</code>      | MySQLDB       | joins/transactions capability profile and MySQL-specific rendering expectations          |
| <code>test_mongodb_connector.cpp</code>    | MongoDB       | negative capability validation and BSON-like filter rendering                            |
| <code>test_redis_connector.cpp</code>      | RedisDB       | absence of joins/transactions/aggregation and key-value command materialisation          |
| <code>test_cassandra_connector.cpp</code>  | CassandraDB   | capability constraints and CQL rendering correlates                                      |
| <code>test_neo4j_connector.cpp</code>      | Neo4jDB       | transaction capability and graph-oriented query materialisation                          |

### 11.3. Integration and live tests

The integration layer is organised around two principles. The first is to have a backend-independent mock baseline that can validate the query-execution contract without external infrastructure. The second is to have live tests for real backends whenever the corresponding targets and environment flags are available.

**Table 22. Catalogue of the integration and live tests**

| File                     | Execution mode                              | What it demonstrates                                                                                   |
|--------------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------|
| test_mockdb.cpp          | always-available<br>integration<br>baseline | query composition, parameter binding, prepared queries, and CRUD scenarios without an external backend |
| test_sqlite.cpp          | local integration<br>test                   | a real relational execution path with capability assertions and hydration scenarios                    |
| test_postgresql_live.cpp | conditional live<br>test                    | real connection management, table lifecycle, and CRUD execution on PostgreSQL                          |
| test_mysql_live.cpp      | conditional live<br>test                    | real MySQL driver path, insert/update/delete, and select filtering                                     |
| test_mongodb_live.cpp    | conditional live<br>test                    | execution of document-oriented scenarios against a live backend                                        |
| test_redis_live.cpp      | conditional live<br>test                    | key-value and hash-command materialisation in a Redis environment                                      |
| test_cassandra_live.cpp  | conditional live<br>test                    | wide-column execution and backend-specific CQL scenarios                                               |
| test_neo4j_live.cpp      | conditional live<br>test                    | graph query execution through a controlled Docker environment                                          |

This catalogue matters because it shows that repository-backed evidence is layered. Not every backend has the same operational cost for live verification, but the repository contains a clear strategy indicating how and when such tests are activated.

## 11.4. Capability evidence by backend family

One of the central ideas of the library is that connectors must not claim more abilities than they actually provide. For that reason, capability evidence is distributed between unit tests and integration tests rather than being left only at the level of documentation.



**Table 23. Examples of capability evidence in the test layer**

| Backend    | Principal test file           | What is demonstrated                                                               |
|------------|-------------------------------|------------------------------------------------------------------------------------|
| SQLite     | test_sqlite.cpp               | presence of supports_joins and supports_transactions in a real integration context |
| PostgreSQL | test_postgresql_connector.cpp | presence of joins, transactions, and aggregation capability tags                   |
| MySQL      | test_mysql_connector.cpp      | presence of joins and transactions, but absence of aggregation capability          |
| MongoDB    | test_mongodb_connector.cpp    | absence of SQL-oriented capability tags and correct filter-rendering behaviour     |
| Redis      | test_redis_connector.cpp      | absence of joins/transactions/aggregation and key-value materialisation semantics  |
| Neo4j      | test_neo4j_connector.cpp      | presence of transactions and graph-specific execution focus                        |
| Cassandra  | test_cassandra_connector.cpp  | restricted capability profile and wide-column contract behaviour                   |

## 11.5. CMake registration as part of the verification design

The build system is not a neutral background, but an active component of verification. This is visible in both tests/unit/CMakeLists.txt and tests/integration/CMakeLists.txt. In the unit layer, all core tests are grouped into a single executable target and registered through gtest\_discover\_tests. In the integration layer, some tests are always present, while live tests are activated only when the corresponding targets and feature flags are available.

```

1 add_executable(test_unit
2     test_types.cpp
3     test_property.cpp
4     test_relationship.cpp
5     test_query.cpp
6     test_mysql_connector.cpp
7     test_postgresql_connector.cpp

```

```

8     test_mongodb_connector.cpp
9     test_redis_connector.cpp
10    test_cassandra_connector.cpp
11    test_neo4j_connector.cpp
12    test_thread_safety.cpp
13    test_wire_protocol.cpp
14    test_cpp26_reflection.cpp
15    test_schema_migration.cpp
16 )
17
18 gtest_discover_tests(test_unit)

```

**Listing 11:** Excerpt from unit-test registration

Listing 11 is short, but highly informative: it describes the unit-verification layer as a systematic catalogue rather than a sporadic set of tests.

```

1 add_executable(test_integration
2     test_mockdb.cpp
3 )
4
5 gtest_discover_tests(test_integration)
6
7 if(TARGET orm::postgresql_live AND ORM_POSTGRESQL_LIVE_ENABLED)
8     add_executable(test_postgresql_live test_postgresql_live.cpp)
9     add_test(NAME PostgreSQLLive COMMAND test_postgresql_live)
10 endif()
11
12 if(TARGET orm::mysql_live AND ORM_MYSQL_LIVE_ENABLED)
13     add_executable(test_mysql_live test_mysql_live.cpp)
14     add_test(NAME MySQLLive COMMAND test_mysql_live)
15 endif()

```

**Listing 12:** Excerpt from conditional live-test registration

Listing 12 shows that integration and live verification are intentionally conditional. This is architecturally sound, because not every backend is always available in every environment, yet the registration framework remains formally specified.

## 11.6. Conclusions from the appendix

The present appendix shows that verification of the library possesses its own internal architecture. It includes a catalogue of unit evidence, a baseline integration execution path, conditional live scenarios, and a build-level mechanism for test registration and activation. This is an important part of the research value of the work, because it makes it possible to connect

the claims of the main text to concrete, verifiable artefacts.

Consequently, the thesis does not merely describe a library design, but documents a system whose principal claims can be traced to source headers, CMake registration, and executable tests. It is exactly this traceability that is critical for evaluating the Compile-time Object-Relational Mapping library as a completed engineering and academic work.

## **XII. ATLAS OF CAPABILITY PROFILES AND BACKEND MATURITY**

The present appendix complements the main analysis from Chapters VI and VII by presenting the capability mechanism and backend maturity in a more synthesised form. If the previous appendix systematised the testing infrastructure, the focus here falls on how the different backends are positioned relative to the shared connector contract.

This type of synthesis is useful for two reasons. First, it makes it possible to view the library as a coordinated system of capability profiles rather than merely a set of isolated adapters. Second, it allows a clearer formulation of what it means for a backend to be minimally supported, operationally useful, or architecturally mature.

### **12.1. The capability mechanism as a formal layer**

The file `lib/include/ORM/connector/capabilities.hpp` defines the capability layer through the primary template `connector_trait<DB>` and the set of capability-tag structures in namespace `cap`. In the version of the library analysed here, this set includes `supports_joins`, `supports_transactions`, `supports_aggregation`, `supports_embedding`, `supports_upsert`, and `supports_bulk_insert`.

What matters is that capabilities are not kept in a runtime registry and are not described as textual metadata. They are encoded at the type level through the presence or absence of nested aliases inside the specialisation of `connector_trait<DB>`. This means that admissibility of an operation can be used as a compile-time condition inside the query API and the associated concepts.

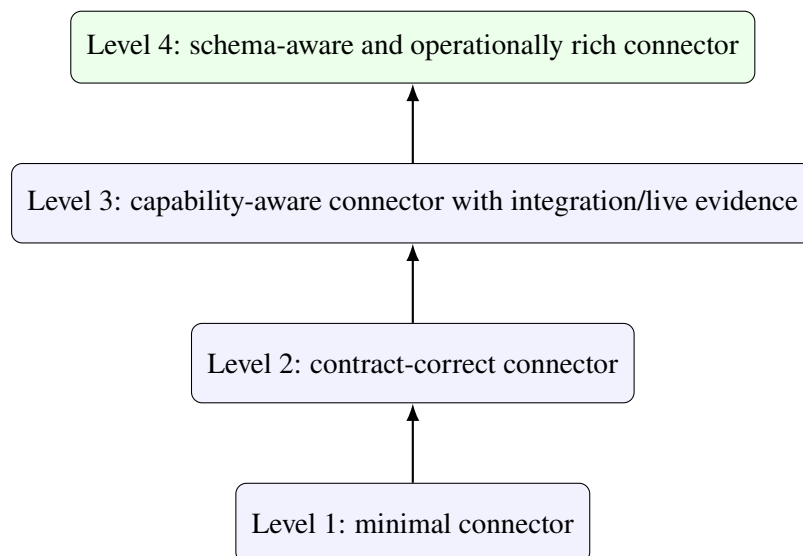
**Table 24. Meaning of the capability tags in the analysed architecture**

| Capability tag                     | Architectural meaning                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------|
| <code>supports_joins</code>        | the backend can materialise join-like composition between logical entity descriptions       |
| <code>supports_transactions</code> | the backend exposes sufficient transactional semantics or API hooks for transaction helpers |
| <code>supports_aggregation</code>  | the backend supports aggregation operations within its corresponding query model            |
| <code>supports_embedding</code>    | the backend admits document-oriented or nested-object materialisation strategies            |
| <code>supports_upsert</code>       | the backend allows one-step update-or-insert behaviour in a suitable form                   |
| <code>supports_bulk_insert</code>  | the backend exposes an efficient path for batch insert scenarios                            |

Table 24 is important because it shows that the capability layer is not decorative. Every tag must have a concrete architectural meaning rather than acting as a purely descriptive label.

## 12.2. Connector maturity as a ladder rather than a binary judgement

In practice, a connector rarely moves directly from “missing” to “fully ready”. It is more accurate to think of it as climbing a ladder of maturity.

**Figure 19. Maturity ladder for backend connectors**

The ladder in Figure 19 allows a more precise evaluation of backend integrations:

1. **Minimal connector** — the backend has a `connector_trait<DB>` specialisation and can at least participate in the formal compile-time contract.
2. **Contract-correct connector** — `is_connector` evidence and unit tests exist for capability honesty and baseline materialisation logic.
3. **Capability-aware connector with execution evidence** — integration or live tests are present, showing that the capability profile is not merely declarative.
4. **Schema-aware and operationally rich connector** — the backend can participate in migration, thread safety, prepared-query workflows, and broader production-oriented scenarios.

### 12.3. Capability profiles by backend family

By synthesising the unit and integration tests, it becomes possible to form a compact capability matrix for the principal backends.

**Table 25. Capability profiles of the principal backends according to available tests and connector specialisations**

| Backend    | Joins       |             |                         | Upsert      | Bulk insert | Short comment                                                                 |
|------------|-------------|-------------|-------------------------|-------------|-------------|-------------------------------------------------------------------------------|
|            | Transaction | Aggregation |                         |             |             |                                                                               |
| SQLite     | present     | present     | present                 | limited     | limited     | the closest full relational reference path in the repository                  |
| PostgreSQL | present     | present     | present                 | specific    | possible    | strong relational contract with live execution evidence                       |
| MySQL      | present     | present     | absent in current tests | specific    | possible    | operational path exists, but the capability profile remains more conservative |
| MongoDB    | absent      | absent      | absent                  | analogue    | not central | strong document model, but no SQL-style capability profile                    |
| Redis      | absent      | absent      | absent                  | analogue    | limited     | key-value semantics with a minimalist but honest contract profile             |
| Neo4j      |             | present     | absent                  | not central | not central | graph backend with transaction focus rather than SQL-style aggregation        |
| Cassandra  | absent      | present     | absent                  | not central | possible    | wide-column backend with a restricted yet formally defined capability set     |

Table 25 should not be read as a universal truth about the databases themselves. It describes the specific architectural position of the library toward these backends in the analysed version of the repository.

## **12.4. Maturity map by available test evidence**

The capability profile alone is not sufficient. From the standpoint of engineering reliability, it matters whether a backend has only contract-level unit tests, local integration scenarios, or live execution evidence as well.



**Table 26. Backend maturity map according to the available evidence base**

| <b>Backend</b> | <b>Maturity level</b> | <b>Principal evidence base</b>                                      | <b>Interpretation</b>                                                        |
|----------------|-----------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------|
| MockDB         | level 3               | test_mockdb.cp<br>p                                                 | serves as the backend-independent execution baseline for the query contract  |
| SQLite         | level 3 to 4          | test_sqlite.cp<br>p                                                 | local real execution path with rich query evidence and capability assertions |
| PostgreSQL     | level 3               | test_postgresq<br>l_connector.cp<br>p, test_postgre<br>sql_live.cpp | strong contract plus conditional live backend verification                   |
| MySQL          | level 3               | test_mysql_c<br>onconnector.cpp,<br>test_mysql_liv<br>e.cpp         | real operational path with a more conservative capability set                |
| MongoDB        | level 3               | test_mongodb_c<br>onconnector.cpp,<br>test_mongodb_l<br>ive.cpp     | document-oriented execution evidence without SQL-style capabilities          |
| Redis          | level 3               | test_redis_c<br>onconnector.cpp,<br>test_redis_liv<br>e.cpp         | key-value execution evidence with a limited but honest contract profile      |
| Neo4j          | level 3               | test_neo4j_c<br>onconnector.cpp,<br>test_neo4j_liv<br>e.cpp         | graph execution path with Docker-gated live verification                     |
| Cassandra      | level 3               | test_cassand<br>ra_connector.c<br>pp, test_cassa<br>ndra_live.cpp   | wide-column path focused on formal suitability and live coverage             |

## **12.5. What the capability atlas reveals about the architecture**

Three practical conclusions follow from the capability atlas. First, the library does not attempt to impose artificial uniformity on the backends. On the contrary, it makes differences explicit through capability tags and contract-level tests. Second, the relational backends naturally concentrate a richer capability profile, while the document, graph, and key-value backends are assessed through different and more appropriate criteria. Third, connector maturity should be measured not only by the number of supported operations, but also by the existence of real evidence.

These conclusions are important from a research point of view as well. They show that the compile-time ORM architecture is viable precisely because it does not erase the differences between storage models. Instead, it formalises and controls them at the level of types, capability profiles, and verification strategies.

## **12.6. Conclusion**

The present appendix completes the synthesis between the connector contract and the empirical evidence layer. If the main text showed how the library works, the capability atlas shows how it structures the diversity of backends in a disciplined and verifiable way. This is yet another reason to treat the work not merely as a software library, but as a formally organised engineering system.